ROS2 系统学习记录

爱吃烧饼的大熊猫

2026 年 10 月 22 日

目录

前言 **vii**

第一部分　基础篇 **1**

第一章　Ubuntu、WSL 与 ROS 简介 **3**

1.1　什么是 Ubuntu 3
1.1.1　从 Linux 说起 3
1.1.2　为什么机器人开发普遍使用 Ubuntu 3
1.2　在 Windows 上使用 Ubuntu：WSL 4
1.2.1　WSL 是什么 5
1.2.2　和装一台真机 Ubuntu 相比 6
1.3　安装并配置 WSL 6
1.3.1　先看清楚自己装的是哪一版 6
1.3.2　从 2.7 跳到 3.0：这次升级改了什么 7
1.3.3　连接要等好久，是哪里慢 8
1.3.4　安装 Ubuntu：桌面版 ISO 根本用不上 9
1.3.5　把 WSL 挪到 D 盘 11
1.4　中文显示和中文输入 12
1.4.1　第一课：apt 必须 sudo 12
1.4.2　第二课：包名对不上，整条命令全废 13
1.4.3　第三课：语言包和 locale 13
1.4.4　第四课：输入法，别装 14
1.5　wsl –shutdown 到底敲在哪 14
1.6　gedit 后面那串奇怪字符 14
1.7　在 WSLg 里跑出一整个 GNOME 桌面 15
1.7.1　两条路，选哪条 15
1.7.2　准备工作 16
1.7.3　让 GNOME Shell 用正确的方式启动 16

1.7.4 排错一：snap 卡在 0%，然后把 dpkg 搞坏了 17
1.7.5 排错二：asserts failed 18
1.7.6 桌面跑起来之后 19
1.8 ROS 是什么 19
1.8.1 ROS 不是操作系统 19
1.8.2 核心概念：节点与通信 20
1.8.3 从 ROS 1 到 ROS 2 20
1.9 ROS 1 与 ROS 2 的主要区别 21
1.10 ROS 2 与 Ubuntu 版本的搭配 21
1.11 在 WSL 里安装 ROS 2 21
1.11.1 官方路线：ros2-apt-source 22
1.11.2 国内镜像路线 22
1.11.3 安装并验证 23
1.11.4 如果两个节点互相听不见 24
1.12 让 rviz2 正常出图 24
1.12.1 第一招：强制走 XCB 24
1.12.2 第二招：补图形库，并让 Mesa 用上显卡 25
1.12.3 最后一招：软件渲染兜底 26
1.13 这一路的坑，一张表收尾 26
1.14 这条路值不值得走 27
1.15 本书的学习路线 28

第二章 Ubuntu 常用指令 29
2.1 打开终端 29
2.2 Linux 中的常用命令 29
2.2.1 ls 命令 29
2.2.2 cd 命令 30
2.2.3 pwd 命令 31
2.2.4 mkdir 命令 32
2.2.5 rm 命令 33
2.2.6 cp 命令 33
2.2.7 mv 命令 34
2.2.8 cat 命令 34
2.2.9 more 命令 35
2.2.10 less 命令 35
2.2.11 head 命令 36
2.2.12 tail 命令 36

2.2.13 which、whereis、locate、find 命令 36
2.2.14 grep 命令 37
2.2.15 wc 命令 38
2.2.16 chmod 命令 38
2.2.17 tar 命令 39
2.2.18 ps 命令 40
2.2.19 top 命令 40
2.2.20 free 命令 41
2.2.21 df 命令 41
2.2.22 du 命令 41
2.2.23 ln 命令 42
2.2.24 date 与 cal 命令 42
2.2.25 kill 命令 43
2.3 鱼香 ROS 的指令 43
2.4 用自然语言来使用 Unbuntu 44

第三章 Ubuntu 环境下编程 45
3.1 Hello weijianguizhou! 45
3.1.1 C++ 45
3.1.2 python 46
3.2 什么是编译执行和解释执行 47

第四章 ROS2 基础 49
4.1 ROS2 的下载 49
4.2 ROS2 体验 50
4.2.1 你说我听 50
4.2.2 涂鸦乌龟 51
4.2.3 RQT 可视化 52
4.3 ROS2 系统架构 52
4.3.1 分层架构总览 52
4.3.2 操作系统层 53
4.3.3 DDS 实现层 53
4.3.4 中间件抽象层 RMW 54
4.3.5 客户端库 RCL 与语言绑定 54
4.3.6 应用层 55
4.4 ROS2 中间件 DDS 架构 55
4.4.1 中间件 55

4.5 DDS 和 ROS2 架构 56
4.6 DDS 通信模型 57
4.7 ROS2 的节点 57
4.7.1 使用 g++ 编译 ROS2 节点 57

前言

第一次接触或者说听说 ROS 是在 2025 年 12 月，那时候还在兵器学科楼 340，坐在办公室最东侧的一个小桌子上。刘方鑫师兄还没毕业，帮着师兄审核毕业论文，一起聊天，好不快乐。那时候师兄给我讲讲 MPC 和 WBC，一起去找张老师聊动力学控制，真的觉得这样的日子要是能一直持续下去就好了。

眨眼来到师兄毕业去湖南株洲工作前夕，师兄把他电脑上的代码全部移到了我的电脑上，那是我第一次接触 Unbuntu，第一次接触 ROS。

第二次接触 ROS 就是这次了，这次得从头开始学了。第一次接触是师兄弟间的闲聊，这一次就是朋友间的承诺了。希望这次可以利用好闲散的时间，好好把 ROS2 学好，学通。

第一部分

基础篇

第一章　Ubuntu、WSL 与 ROS 简介

在开始敲第一行命令之前，先用一章的篇幅把几个名字弄清楚：Ubuntu 到底是什么，为什么机器人开发几乎离不开它；WSL 又是什么，为什么它能让 Ubuntu 直接住在 Windows 里；ROS 又是什么，为什么它叫操作系统却不是操作系统。这一章既讲概念，也把环境完整搭起来：装好 WSL、配好中文、跑通图形界面，最后把 ROS 2 装上并验证。读完这一章，后面的每一章就都有了一块可以落脚的地基。

1.1　什么是 Ubuntu

1.1.1　从 Linux 说起

先纠正一个常见的误解：我们平时说的 “Linux”，严格来讲只是操作系统的一个内核，也就是负责管理硬件、进程、内存的那部分最核心的程序。光有内核还不足以日常使用，还需要配上图形界面、各类软件和工具，才构成一个完整可用的操作系统。内核加上这些软件的整体，叫作 Linux 的*发行版*。

Ubuntu 就是众多发行版中人气最高、资料最全的一个。它基于另一个老牌发行版 Debian，由英国的 Canonical 公司维护，完全免费、开放源代码。因为用的人多，网上的教程、第三方软件、硬件驱动的支持都最齐全，所以它也是初学者最合适的入门选择。

Ubuntu 的版本号很有规律，形如 “年份. 月份”。比如 22.04 表示 2022 年 4 月发布。它每年发布两次，其中*偶数年的 4 月*会发布一个特别版本，名字里带 LTS（Long Term Support，长期支持），官方会提供长达五年的维护更新。机器人开发领域几乎清一色使用 LTS 版本，图的就是稳定和长期可用，本书也以 LTS 版本为前提。

1.1.2　为什么机器人开发普遍使用 Ubuntu

机器人开发选择 Ubuntu，通常出于几个现实原因：

- **ROS 官方的主战场就是 Ubuntu**。ROS 的安装、文档、示例在 Ubuntu 上支持得最好，很多版本组合甚至只有 Ubuntu 这一条 “官方推荐” 路线，跟着走最不容易踩坑。

- **免费开源**。机器人项目可以自由使用、修改和分发，不必为许可证操心。
- **生态成熟**。激光雷达、相机、机械臂等硬件厂商大多优先提供 Linux 驱动；论文开源代码、算法库也常在 Ubuntu 上测试。
- **命令行与脚本化能力强**。机器人常常需要远程登录、批量处理、自动运行，Ubuntu 的命令行环境让这些操作非常顺手。这正是第二章要练的基本功。

当然，ROS 并不是只能在 Ubuntu 上跑，其他 Linux 发行版甚至 Windows 也可以。但对新手而言，跟随 ROS 官方支持最好的 Ubuntu，能省下大量和环境搏斗的时间。

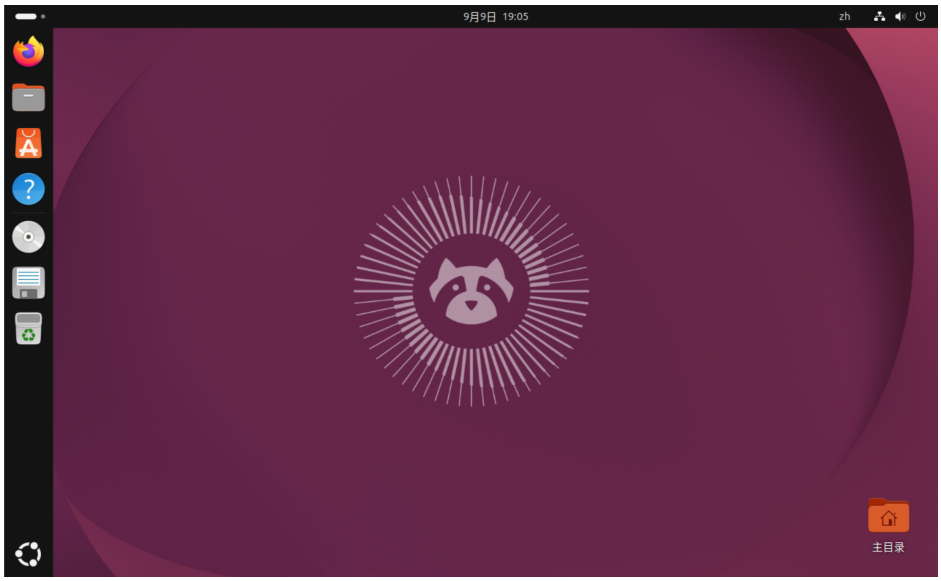


图 1.1: Ubuntu 桌面界面

1.2 在 Windows 上使用 Ubuntu：WSL

本书的最终目标，是让 ROS 2 在一套 Ubuntu 环境里跑起来。多数读者日常使用的电脑装的是 Windows，直接改装系统代价太大，装双系统又要重启切换，也谈不上方便。更轻的起步方式是：用 Windows 自带的 **WSL**，让 Ubuntu 直接住在 Windows 里。

WSL 全称 Windows Subsystem for Linux，中文叫“适用于 Linux 的 Windows 子系统”。一句话解释：让 Linux 直接住在 Windows 里，不必再装一台虚拟机。它开机一两秒就能进系统，能直接读写 Windows 里的文件，图形程序也能开在 Windows 桌面上。

不过这条路上的坑明显更密，几乎每走一步都会撞上一个报错。所以本章后面的环境部分不按教科书的顺序写，而是按照笔者当时真实的踩坑顺序写：先被什么

卡住，屏幕上报了什么样的一段话，最后又是怎么过的。你可以把它当成一份通关记录，遇到同样的报错时直接翻到对应的小节抄命令。

1.2.1 WSL 是什么

严格说来，WSL 1 只是把 Linux 的系统调用“翻译”成 Windows 能听懂的形式，兼容性一般。从 WSL 2 开始，微软干脆在 Windows 内部塞了一台轻量虚拟机，里面跑的是真正的 Linux 内核。所以今天用的 WSL 2 和 WSL 3，本质上是“一台你看不见、也不用管的虚拟机”。

这句话很重要，因为它提前解释了后面一堆坑的共同来源：你其实是在两套系统之间来回横跳，而两边的命令、路径、权限规则全都不一样。

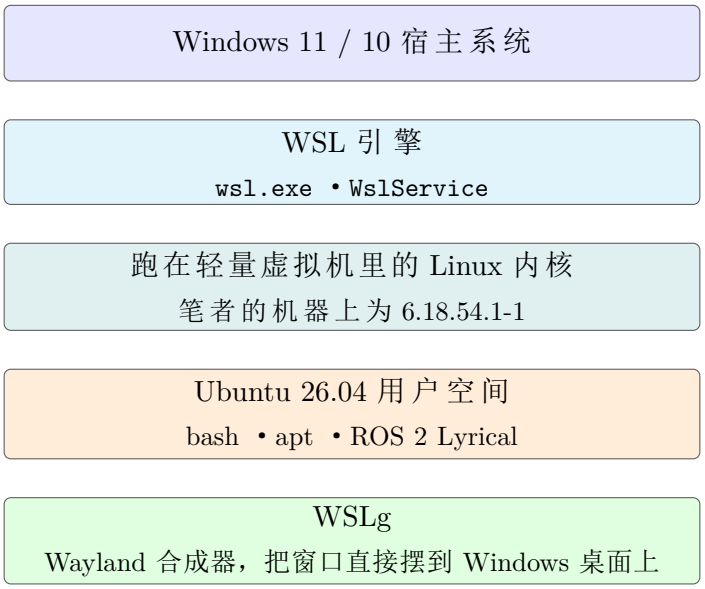


图 1.2: WSL 的分层结构。Windows 里先有一层 WSL 引擎，引擎之下的 Linux 内核跑在轻量虚拟机中，内核之上才是发行版和你的程序；WSLg 负责把图形窗口送回 Windows 桌面

从图 1.2 自上往下看，我们的操作其实分属不同楼层。最塌实的做法，是先记住哪条命令该在哪一层敲：

在 Windows 的 PowerShell 里敲	在 WSL 的 Ubuntu 里敲
`wsl -l -v`	`sudo apt update`
`wsl --update --web-download`	`systemctl --user ...`
`wsl --install --from-file x.wsl`	`ros2 run turtlesim ...`
`wsl --manage Ubuntu --move D:\WSL`	`glxinfo -B`
`wsl --shutdown`	`wsl.exe --shutdown`（唯一的特例）

本章后面反复出现的报错，绝大多数都只是“在错误的楼层敲了正确的命令”而已。

1.2.2 和装一台真机 Ubuntu 相比

WSL 并不是唯一的选择，把 Ubuntu 直接装到硬盘上（单系统或双系统）也是一条路。两者差别如下：

对比项	真机安装 Ubuntu	WSL
本质	一台完整的电脑	Windows 里的轻量 Linux 子系统
磁盘占用	几十 GB 起	几个 GB 起
开机速度	重启并切换系统	一两秒
和 Windows 传文件	靠分区挂载或网络共享	直接访问 `/mnt/c`
图形界面	完整桌面，装完就有	单个程序开箱即用，完整桌面要折腾
快照回滚	可用备份工具整机还原	没有，弄坏了只能重装

笔者的判断是：只想跟着教程跑通 ROS 2、写几个节点，WSL 完全够用，而且比装双系统顺手得多；如果想要一台“随便怎么折腾都能救回来”的机器，那就得自己养成导出备份的习惯（本章后面会讲 `wsl --export`），或者干脆用一台独立的机器。

1.3 安装并配置 WSL

1.3.1 先看清楚自己装的是哪一版

一切从两条命令开始。第一条看引擎版本，第二条看装了哪些发行版：

查看 WSL 版本与已安装的发行版（在 PowerShell 中执行）

```
wsl -v
wsl -l -v
```

第一条命令会打印一大串版本号，刚看的时候一头雾水。按字段拆开就好理解了：

字段	含义
WSL 版本	引擎本身的版本，升级它用 `wsl --update`
内核版本	那台轻量虚拟机里跑的 Linux 内核版本
WSLg 版本	图形界面组件的版本
MSRDC / Direct3D / DXCore	图形转发相关的组件
Windows 版本	宿主系统版本

第二条命令的输出，常常是新手遇到的第一个“惊吓”：

什么都没装时的提示

```
适用于 Linux 的 Windows 子系统没有已安装的分发。
可按照以下说明安装分发来解决此问题:
使用 'wsl.exe --list --online' 列出可用的分发
和 'wsl.exe --install <Distro>' 进行安装。
```

这其实不是报错。前面说过，WSL 引擎只是一个“壳”，Ubuntu、Debian 这些发行版是要另外装的“整车”。引擎升级和装发行版是两件独立的事，先装引擎、后装发行版完全正常。

笔者的经历就是一次典型的时间差：早上敲 `wsl -v`，显示的是 `2.7.8.0`，内核 `6.18.33.1-1`；晚上再敲，已经变成 `3.0.3.0`、内核 `6.18.54.1-1` 了。中间什么都没干，只是 Windows 自动更新顺手把 WSL 推上去了。

1.3.2 从 2.7 跳到 3.0：这次升级改了什么

从 2.7.8 一步跨到 3.0.3，看着像一次大改，其实不然。WSL 3 并不是 WSL 1 到 WSL 2 那种底层架构革命，它更像是把原本处于预览状态的容器功能正式转正，顺便把大版本号也抬了一级。

几个值得一提的变化：

- **原生 Linux 容器**。新增 `wslc` 命令行工具（WSL Containers），可以不装 Docker Desktop 就在 Windows 上创建、运行 Linux 容器，一行命令就能起一个 Nginx。容器也不再依赖某个特定的发行版，可以独立运行。
- **文件访问变快**。默认启用 virtiofs 文件系统，取代了沿用多年的 Plan 9 协议，访问 Windows 侧文件的性能官方声称最高可以翻一倍。
- **新的网络模式**。引入了 consomme 模式，Linux 的流量经由 Windows 主机转发，对代理服务器和企业网络策略的兼容性更好。
- **内核换代**。3.0 基于 Linux 6.18，并增加了自动虚拟 NUMA 支持，能在几百个线程的大型多路服务器上稳定跑。
- **实验性稀疏 VHD**。从 3.0.2 开始可以开启稀疏虚拟硬盘，磁盘空间按实际写入量占用，能省不少地方。

笔者用的 3.0.3 还是一个预发布（Pre-release）版本，在容器功能转正的基础上，又带了内核升级到 6.18.54.1、修复 `LifetimeManager` 竞态条件、改进 `wslc` 事件过滤器等一批稳定性修补。

升级命令只有一句，建议加上`--web-download`，因为默认的走法在某些网络环境下会卡住不动：

升级 WSL（在 PowerShell 中执行）

```
wsl --update --web-download
```

1.3.3 连接要等好久，是哪里慢

装完之后的第一感受往往不是“真快”，而是“怎么这么卡”。笔者当时的原话就是三个字：为啥连接要好久。

这件事不能笼统地问，得先判断慢在哪一环。两类情况，病因完全不同：

- **启动就慢**：敲完 `wsl` 回车，光标要干等十几秒才出现 Linux 提示符。这属于启动初始化问题。
- **启动不慢，联网慢**：很快就进系统了，但是 `ping`、`curl`、`apt update` 一动就卡很久。这基本是网络或者 DNS 的问题。

启动初始化慢

两个容易被忽略的原因：

- **断掉却还挂着的网络驱动器**。如果曾经映射过公司的 SMB 共享盘，而现在并不在同一个网络里，Windows 和 WSL 在启动时会反复尝试连接它，卡顿非常明显。把用不到的网络驱动器映射断开即可。
- **代理软件或 VPN 的虚拟网卡**。某些代理工具的虚拟网卡会和 WSL 的虚拟网络抢占资源，拖慢初始化。可以先把代理关掉试一次，快慢对比一下就清楚了。

联网慢：网络模式与 DNS

WSL 支持几种网络模式，默认的 NAT 模式在国内网络下经常出问题。改写配置文件 `C:\Users\<你的用户名>\.wslconfig`：

切换为 mirrored 网络模式

```
[wsl2]
networkingMode=mirrored
```

mirrored 模式让 WSL 直接“镜像”宿主机的网络接口，能更好地兼容主机上的代理，很多莫名其妙的连接问题会随之消失。如果某个端口在 mirrored 下还是断断续续，可以再试试前面提到的 consomme 模式。

DNS 是另一个高频嫌疑。每个网络请求之前都要先解析域名，解析一慢，整体感觉就是“网卡了”。有两种处理方式：

- **简单做法**：把 Windows 主机的 DNS 换成公共 DNS（比如 8.8.8.8、1.1.1.1），WSL 通常会自动继承。
- **彻底做法**：在 WSL 内部关掉自动生成，手动指定。编辑`/etc/wsl.conf` 加入 `generateResolvConf = false`，再往`/etc/resolv.conf` 里写一行 `nameserver 8.8.8.8`。

还有一种比较冷门的原因：WSL 虚拟网卡的“大量发送卸载”（LSO）在个别网络环境下反而拖后腿。用管理员权限的 PowerShell 关掉它：

禁用 WSL 虚拟网卡的 LSO

```
Disable-NetAdapterLso -Name "vEthernet (WSL)" -IncludeHidden
```

如果同时还在用 Docker Desktop，那 WSL 卡顿就更常见了，两者网络栈容易打架。可以到“任务管理器 → 服务”里找到 `WslService` 右键重启，往往立刻见效。

最后一条与网络无关，但几乎是所有人都会犯的：**项目文件别放在 `/mnt/c` 下面**。那是 Windows 的文件系统，在它上面跑 `npm install`、编译、批量读写都会慢得离谱。把项目放在 WSL 自己的家目录（比如 `~/projects`），再用 VS Code 的 WSL 扩展打开，速度完全是两个世界。

改了哪个文件	生效方式
`.wslconfig`	在 PowerShell 里执行 `wsl --shutdown`
`/etc/wsl.conf`	同上
`/etc/resolv.conf`（手动维护的那种）	同上

1.3.4 安装 Ubuntu：桌面版 ISO 根本用不上

发行版列表拉不下来，是这条路上第一个真正的拦路虎：

拉取发行版列表失败

```
wsl --list --online
未能从"https://raw.githubusercontent.com/microsoft/WSL/master/distributions/
    DistributionInfo.json"
提取分发列表。错误的网关(502)。
错误代码： Wsl/HTTP_E_STATUS_BAD_GATEWAY
```

这个 502 跟 WSL 本身没什么关系。它只是说明：你的电脑去 GitHub 取那份发行版清单时，被中间的网关拦了回来。所以从易到难依次试：

1. 换用 `wsl --update --web-download` 升级一次，有时候顺带就好了；
2. 关掉代理或 VPN 再试，很多时候罪魁祸首正是它；

3. 换 DNS，或者干脆往 `hosts` 文件里手写 `raw.githubusercontent.com` 的 IP；
4. 从微软商店直接搜 Ubuntu 安装，这条路完全绕开 GitHub，最稳；
5. 离线安装，也就是下一小节要讲的办法。

为什么 ISO 不能直接用

笔者的第一反应很自然：我手里不是有 `ubuntu-26.04.1-desktop-amd64.iso` 吗，直接装不就完了。结果是不行，而且不行得有道理。

桌面版 ISO 是一张“安装光盘”。它的内容是启动环境加一个图形化安装程序，运行起来之后要一步一步问你装到哪块硬盘、要不要格式化。而 WSL 要的东西完全不同：它需要的是一套已经装好、解压即可运行的根文件系统，直接挂起来就开机，中间没有“安装”这个步骤。

所以命令也不一样。WSL 用的是`--from-file` 或者`--import`，喂给它的必须是`.wsl` 镜像或者 rootfs 的 tar 包，不是 ISO。

正确的离线安装方式

Ubuntu 官方为每个 LTS 版本都准备了专门给 WSL 打包的镜像，文件名形如 `ubuntu-26.04.1-wsl-amd64.wsl`，几百 MB，在 `releases.ubuntu.com` 或者国内镜像站都能找到。下载好之后：

从.wsl 镜像离线安装（在 PowerShell 中执行）

```
wsl --install --from-file "C:\Users\john\Downloads\ubuntu-26.04.1-wsl-amd64.
    wsl"
```

安装过程要解包并注册文件系统，会等上一两分钟。完成后会提示你创建一个 UNIX 用户名和密码，跟着走就行。

如果版本不够新、不认`--from-file` 这个参数，可以改用等价的导入方式（`.wsl` 本质上就是一个 tar 归档）：

备选方案：手动导入

```
mkdir C:\Users\john\WSL\Ubuntu-26.04
wsl --import Ubuntu-26.04 C:\Users\john\WSL\Ubuntu-26.04 "C:\Users\john\
    Downloads\ubuntu-26.04.1-wsl-amd64.wsl" --version 2
```

两者有个重要区别：**用 `--import` 装出来的系统默认以 root 登录**，进去之后还得自己建一个普通用户。用`--from-file` 则会在首次启动时引导你创建用户，省事得多。

装完验证一下，看到版本号为 2 就对了：

验证安装结果

```
wsl -l -v
```

1.3.5 把 WSL 挪到 D 盘

WSL 的磁盘文件（ext4.vhdx）默认塞在 C 盘的用户目录深处，用着用着 C 盘就见红了，所以迟早要挪走。先看两种办法。

方法一：直接移动（首选）

新版本的 WSL 提供了官方移动命令，一步到位，不产生中间备份文件：

用 –move 迁移发行版（在 PowerShell 中执行）

```
wsl --shutdown
wsl --manage Ubuntu-26.04 --move D:\WSL\Ubuntu-26.04
```

迁移完可以确认一下注册表里记的路径是不是已经改过去了：

查看发行版的实际存放位置

```
(Get-ChildItem HKCU:\Software\Microsoft\Windows\CurrentVersion\Lxss |
  Where-Object { $_.GetValue("DistributionName") -eq 'Ubuntu-26.04' }).
    GetValue("BasePath")
```

返回 D:\WSL\Ubuntu-26.04 就说明成功了。如果报 ERROR_FILE_NOT_FOUND 之类的错误，说明注册表没更新干净，那就改用方法二。

方法二：导出再导入（最稳）

这个办法的原理是 “先打包备份，再解包到新位置”，兼容性最好，也最让人放心：

导出、注销、再导入

```
mkdir D:\WSL_Backup
wsl --export Ubuntu-26.04 D:\WSL_Backup\ubuntu-backup.tar
wsl --unregister Ubuntu-26.04
mkdir D:\WSL\Ubuntu-26.04
wsl --import Ubuntu-26.04 D:\WSL\Ubuntu-26.04 D:\WSL_Backup\ubuntu-backup.
    tar
```

注意中间那一步--unregister 会真的把 C 盘上的原系统删掉，执行之前一定确认备份文件已经生成好了。导入之后同样会变成 root 登录，需要重新配置默认用户。

如果还没装、想一步到位装在 D 盘，安装时直接带上`--location` 参数即可：

全新安装并指定位置

```
wsl --install -d Ubuntu-26.04 --location D:\WSL\Ubuntu-26.04
```

一个特别典型的误会

笔者当时在 WSL 里敲了这么一行，然后被权限拒绝：

错误示范

```
mkdir /opt/WSL
mkdir: Permission denied
```

表面上是权限问题（`/opt` 是系统目录，普通用户写不进去，需要 `sudo`），但真正的问题是层次搞错了：`/opt/WSL` 是 **Linux 文件系统里**的一个目录，它实际存在 C 盘那个 `ext4.vhdx` 里面，跟 Windows 的 D 盘一点关系都没有。想迁移到 D 盘，必须回到 PowerShell 那边去操作。

判断自己要做哪件事，看这张表就够了：

你想做的事	在哪敲	命令
在 Linux 里建个文件夹	WSL 的 bash	`sudo mkdir /opt/WSL`
把整个发行版迁到 D 盘	Windows PowerShell	`wsl --manage ... --move ...`
访问 D 盘上的文件	WSL 的 bash	直接进 `/mnt/d/...`

1.4 中文显示和中文输入

系统装好、能进 bash 了，下一步自然是让它能好好显示中文。这件事其实分三层，很多人把三层混在一起，于是装了一堆互相打架的包。

1. **locale**：决定系统用什么语言、什么编码，关系到日期格式、排序、提示信息；
2. **字体**：决定汉字有没有字形可以画出来，缺字体就显示成方块；
3. **输入法**：决定你怎么把中文敲进去。

1.4.1 第一课：apt 必须 sudo

拦在第一道的是权限，而且报错信息长得很唬人：

apt 的权限报错

```
Error: Could not open lock file /var/lib/dpkg/lock-frontend - open (13:
   Permission denied)
```

```
Error: Unable to acquire the dpkg frontend lock (/var/lib/dpkg/lock-frontend
    ), are you root?
```

意思很直白：装软件要动/var/lib/dpkg 这类系统目录，普通用户没有权限，拿不到那把“锁”。命令前面加 sudo 就好。

1.4.2 第二课：包名对不上，整条命令全废

这条坑很值得单独说。笔者当时一口气装了一串包：

看起来没问题的命令

```
sudo apt install xfonts-wqy xfonts-unifont fonts-wqy
```

结果只有一句 Unable to locate package fonts-wqy。看起来只是少装一个包，其实不是：**apt 只要在一条命令里遇到一个找不到的包，整条命令就会整体失败**。也就是说 xfonts-wqy 和 xfonts-unifont 一个都没装上。

原因是 Ubuntu 里压根没有叫 fonts-wqy 的包，文泉驿字体是拆开成好几个子包的。正确的写法是：

分两条，先装 X 字体，再装真正有用的字体

```
sudo apt install xfonts-wqy xfonts-unifont
sudo apt install fonts-wqy-zenhei fonts-wqy-microhei fonts-noto-cjk
```

顺带解释一下这两类的区别：xfonts-* 是旧式的 X 核心字体，对 gedit 这类现代 GTK 程序帮助很有限；真正起作用的是 fontconfig 体系下的字体，其中 fonts-noto-cjk 是显示中文的主力，比文泉驿完整得多。

1.4.3 第三课：语言包和 locale

语言包提供的是“语言数据”，管的是菜单、对话框有没有中文翻译：

安装中文语言包并设定默认 locale

```
sudo apt update
sudo apt install language-pack-gnome-zh-hans language-pack-zh-hans
sudo update-locale LANG=zh_CN.UTF-8
```

有两点要留意。一是 language-pack-kde-zh-hans 在 Ubuntu 官方源里通常找不到，它属于 Kubuntu 的源，直接装会报错，用 GNOME 的话压根不需要。二是 update-locale 写进的是下次登录才读的配置，当前这个终端不会立刻变，退出重进或者 wsl --shutdown 之后才生效，用 locale 命令可以确认。

1.4.4 第四课：输入法，别装

这是笔者绕得最远的一个弯。当时想当然地装了一整套 fcitx：`fcitx`、`fcitx-config-gtk`、`fcitx-sunpinyin`、`fcitx-googlepinyin`，结果发现是个反方向的努力。

因为 **WSLg 已经把 Windows 的输入法桥接进来了**。在 WSL 里运行的 Linux 图形程序，直接按 Windows 自己的输入法切换键就能打中文，根本不需要在 Linux 内部再养一套输入法。反过来，装了 fcitx 还会跟 WSLg 的桥接机制抢键盘，弄不好两边都失效。

所以结论就一句：输入法一个都不要装。要装的只有字体和语言包。

1.5 wsl –shutdown 到底敲在哪

改完 locale，按照惯例要重启 WSL 让配置生效。于是笔者在 Ubuntu 的 bash 里敲了：

一个必然失败的尝试

```
wsl --shutdown
Command 'wsl' not found, but can be installed with:
sudo apt install wsl
```

两个要点。

第一，`wsl` 是 Windows 侧的命令，你现在在 Linux 里，当然找不到。正确做法是在 PowerShell 或 CMD 里敲。

第二，那句“可以 sudo apt install wsl”是个陷阱，**千万别照做**。Ubuntu 软件源里确实有个叫 `wsl` 的包，但它跟微软的 WSL 毫无关系，装上去什么用都没有，只会让以后查问题时多一层混淆。

如果实在不想切窗口，还有个取巧的办法：WSL 支持直接调用 Windows 的可执行文件（这叫 interop），所以带上 `.exe` 后缀就能在 bash 里执行：

在 Linux 里调用 Windows 命令

```
wsl.exe --shutdown
```

只是这么一来，当前会话会被自己关掉，终端直接退出。另外，如果目的只是让新配置生效，其实用不着 shutdown，把终端窗口关掉再开一个就够了。

1.6 gedit 后面那串奇怪字符

启动 gedit 之后，终端里喷出来一堆这样的东西：

终端里刷出来的一串控制序列

```
gedit;28;13;1;32;1_;28;13;0;32;1_;14;8;1;32;1_;14;8;0;32;1_;
```

看着像乱码，其实不是。这些是应用程序发给终端的 **ANSI 控制序列**，本意是控制光标位置、颜色、清屏之类的动作，正常情况下应该被终端 “解析并执行”，而不是打印出来。之所以原形毕露，是因为 gedit 察觉到自己接的不是一个真正的交互式终端，于是干脆把原始指令直接输出了。

它既不影响使用，也不说明程序坏了。嫌吵的话，把输出丢掉或者让它脱离当前会话就行：

两种清静的办法

```
gedit 2>/dev/null
setsid gedit
```

真正需要警惕的是另一种情况：如果 gedit 窗口里的汉字变成了方块，那才是字体没装好，回去补 `fonts-noto-cjk`。

1.7 在 WSLg 里跑出一整个 GNOME 桌面

到这里为止，WSL 已经能跑单个图形程序了。但笔者贪心，想要一整个 Ubuntu 桌面，跟真机装完系统那样。这件事能做成，但要做好折腾一下午的心理准备。

先说一个必须绕开的坑：从网上找到项目地址，直接把 URL 粘到终端里回车。

把网址当命令敲

```
https://github.com/nufeng1999/WSL_GNOME
-bash: https://github.com/nufeng1999/WSL_GNOME: 没有那个文件或目录
```

Shell 不认识 `https://` 开头的东西，网址不是可执行程序。要用它，得先 `git clone` 下来，再执行里面的脚本。

1.7.1 两条路，选哪条

方案	原理	维护状态
WSL_GNOME	借助 Cygwin 的 XWin 提供 X 服务	2021 年后基本停更
WSLg 原生	WSLg 的 Wayland 合成器直接跑 GNOME	社区活跃

笔者的 WSL 已经是 3.0.3，自带 WSLg，所以最后走的是右边这条。下面记的就是这条路的完整过程，也算给 Ubuntu 26.04 这个比较新的组合留一份记录。

1.7.2 准备工作

首先要打开 systemd。GNOME 的会话是 systemd 用户服务，没有 systemd 就无从谈起。编辑/etc/wsl.conf：

启用 systemd

```
[boot]
systemd=true
```

改完在 PowerShell 里 wsl --shutdown，重新进来才会生效。

然后装桌面。这里有个关键包 mutter-dev-bin，不能漏：

安装桌面环境

```
sudo apt update
sudo apt install ubuntu-desktop mutter-dev-bin
```

mutter-dev-bin 之所以关键，是因为 Ubuntu 26.04 带的是 GNOME 50，而 GNOME 从 49 之后彻底移除了旧的--nested 参数，改用--devkit 模式来启动嵌套桌面，这个参数正是由 mutter-dev-bin 提供的。少装它，后面的启动命令一定失败。

还有一步容易被忽略：把 GDM 屏蔽掉。GDM 是 Ubuntu 的登录管理器，在 WSLg 下面无法正常工作，反而会挡住后面手动启动的会话。

屏蔽 GDM 并降低渲染要求

```
sudo systemctl mask gdm.service
echo 'export GSK_RENDERER=cairo' >> ~/.bashrc
source ~/.bashrc
```

GSK_RENDERER=cairo 是给 GTK4 应用用的保险，能避开 WSL 下 GPU 渲染导致的卡死和黑屏。

1.7.3 让 GNOME Shell 用正确的方式启动

接下来要覆盖 gnome-shell 的启动命令，强制它走--devkit --no-x11 这条路：

创建 systemd 覆盖文件

```
sudo mkdir -p /etc/systemd/user/org.gnome.Shell@.service.d/
sudo nano /etc/systemd/user/org.gnome.Shell@.service.d/override.conf
```

文件内容是这样，注意第一行 ExecStart= 后面是空的：

override.conf

```
[Service]
ExecStart=
ExecStart=/usr/bin/gnome-shell --devkit --no-x11 --mode=%i
```

那行空的 `ExecStart=` 不是笔误，也绝对不能省。systemd 的规则是：同名的 `ExecStart` 会累加执行，只有先写一个空的把默认值清掉，后面这行才会成为唯一的启动命令。

在 nano 里保存退出的手法是：`Ctrl+O` 写入、回车确认文件名，然后 `Ctrl+X` 退出。

1.7.4 排错一：snap 卡在 0%，然后把 dpkg 搞坏了

装 `ubuntu-desktop` 的过程中，屏幕卡在了这么一行：

卡住的 snap 下载

```
==> Installing the firefox snap
下载 snap "snapd" (28254)，来自频道 "stable"
进度：[  0%] [
```

原因是 Ubuntu 26.04 里的 `firefox` 已经是一个 snap 过渡包，安装时要去 snap 商店现场下载本体，而 snap 在国内没有官方镜像，速度很不稳定。卡在 0% 通常就是它在等一个永远等不到的连接。

最要命的是后续：下载连接被重置，安装脚本失败，整个 `dpkg` 的配置流程就中断了，此后所有 apt 操作都会报依赖损坏。

修复顺序是这样的：

修复被中断的 dpkg，并强制移除坏掉的 firefox

```
sudo dpkg --configure -a
sudo apt --fix-broken install
sudo dpkg --force depends -P firefox
```

然后绕开 firefox 重新装桌面。apt 支持在包名后面加一个减号表示“不要装这个”：

排除 firefox 和 thunderbird 再装桌面

```
sudo apt install ubuntu-desktop firefox- thunderbird-
```

如果确实需要一个能用的 Firefox，可以改用 Mozilla 官方 PPA 提供的传统 deb 版本，这样就完全不走 snap 了：

用 PPA 安装 deb 版 Firefox

```
sudo add-apt-repository ppa:mozillateam/ppa
sudo apt update
sudo apt install firefox
```

另外几条缓解 snap 卡顿的办法也记在这里：

- 重启 snapd 服务：`sudo systemctl restart snapd snapd.socket`；
- 清掉缓存的 snap 文件：`sudo rm -rf /var/lib/snapd/cache/*`；
- 给 snapd 单独配上代理。

1.7.5 排错二：asserts failed

配置都写好了，敲下启动命令，等来的却是这样一句：

启动桌面失败

```
systemctl --user daemon-reload
systemctl --user start gnome-session@ubuntu.target
A dependency job for gnome-session@ubuntu.target failed. See 'journalctl -xe
    ' for details.
```

先别猜，直接看日志：

查日志

```
journalctl --user -u gnome-session@ubuntu.target -u org.gnome.Shell@ubuntu.
    service --no-pager -n 50
```

日志里有价值的是这三行：

日志里的关键三行

```
org.gnome.Shell@ubuntu.service: Starting requested but asserts failed.
Assertion failed for org.gnome.Shell@ubuntu.service - GNOME Shell.
Dependency failed for gnome-session@ubuntu.target - GNOME Session (session:
    ubuntu).
```

`asserts failed` 的意思是：服务启动前有个前置条件没满足，systemd 干脆不让它启动。GNOME Shell 的服务单元里带着一条断言，要求环境变量 `XDG_SESSION_TYPE` 必须是 `wayland`。而在 WSLg 环境下，这个变量没有被正确送进 systemd 的用户会话里，断言自然就挂了。

补上它就行。systemd 用户会话会在启动时读取 `~/.config/environment.d/` 下面的配置，把变量写进去：

补上缺失的会话类型变量

```
mkdir -p ~/.config/environment.d/
nano ~/.config/environment.d/gnome-wayland.conf
```

内容：

gnome-wayland.conf

```
XDG_SESSION_TYPE=wayland
XDG_SESSION_DESKTOP=gnome
XDG_CURRENT_DESKTOP=GNOME
```

然后重载再启动：

重新拉起桌面

```
systemctl --user daemon-reload
systemctl --user start gnome-session@ubuntu.target
```

这一次桌面就真的出来了。事后回看，卡了半天的只是三个环境变量。

1.7.6 桌面跑起来之后

几条经验，都是踩出来的：

- **给它起个别名**。启动命令太长，写进 `~/.bashrc`：`alias startgnome='systemctl --user start` 以后一句 `startgnome` 就能呼出桌面。
- **千万别点“注销”或“重启系统”**。WSL 里没有登录管理器接管，桌面窗口一关就再也不会自己弹回来，还得回终端重新敲启动命令。
- **卡死了就 `wsl --shutdown`**。关掉等几秒再进，比在桌面里点什么都管用。
- **平时可以不启动桌面**。纯命令行开发完全不需要它，桌面是要吃 CPU 和内存的。

1.8 ROS 是什么

1.8.1 ROS 不是操作系统

ROS 的全称是 Robot Operating System，中文常译作“机器人操作系统”。这个叫法很有迷惑性：*ROS 并不是一个操作系统*，它不能独立安装在裸机上，而是运行在操作系统（通常是 Ubuntu）之上的一套*开源机器人软件开发框架*。ROS 本身并不

是一个操作系统，而是可以安装在现在已有的操作系统上（Linux、Windows、Mac）上的软件库和工具集。

为什么机器人软件需要这样一套框架？因为一个稍微像样的机器人程序，往往由很多独立的部分组成：读取激光雷达和相机数据、处理感知、做运动规划、控制电机、与人交互……如果每个模块都自己写一套和别人通信的办法，整个系统会迅速变成一团乱麻。ROS 要解决的正是这个问题：它规定了一套统一的通信机制，并提供了大量现成的工具和功能包，让开发者可以专心写自己的那一部分，其余交给生态。

1.8.2 核心概念：节点与通信

ROS 里有两个必须最先建立印象的概念：

- **节点（node）**：一个可以独立运行的程序模块。整个机器人系统可以看作许多节点协作组成的“网络”。
- **通信方式**：节点之间通过话题（*topic*）、服务（*service*）、动作（*action*）等方式交换消息（*message*），消息就是节点间传递的数据包。

打个比方：传感器节点像是不断播报路况的广播电台（话题），导航节点像是收听广播的司机；而当你需要打车时那种“一问一答”的交互（服务），又和广播完全不同。这套通信机制的细节会在后面的章节展开，这里先记住“节点 + 通信”这个总框架即可。

1.8.3 从 ROS 1 到 ROS 2

ROS 起源于 2007 年斯坦福大学的一个机器人项目，随后由 Willow Garage 公司推动，逐渐发展成全球机器人研究界的事实标准，积累了庞大的生态。不过随着机器人应用走向工业化和复杂化，ROS 1 的一些先天设计问题开始暴露：

- 通信依赖一个中心节点 `roscore`，它一旦宕机，整个系统就瘫痪；
- 以单机为主，多机分布式和实时性支持不足；
- 几乎没有安全机制，难以满足工业级要求。

于是社区从 2014 年前后开始重新设计，2017 年发布了 ROS 2 的第一个正式发行版。ROS 2 不是 ROS 1 的小修小补，而是一次架构级的重写。今天，ROS 1 已经停止维护（最后一个发行版是 2020 年的 Noetic），新项目都应当基于 ROS 2。本书讲的也是 ROS 2。

1.9 ROS 1 与 ROS 2 的主要区别

把上面的要点归纳成一张对比表，方便查阅：

对比项	ROS 1	ROS 2
通信机制	自研 TCPROS/UDPROS	基于 DDS 标准
中心节点	依赖 `roscore`	去中心化，无需主节点
实时性	不支持	可通过 DDS 配置支持
多机分布式	支持较弱	原生良好支持
安全机制	几乎没有	内置（如 SROS2）
支持平台	主要 Linux	Linux/Windows/macOS 等

对学习者而言，这些区别并不需要现在就全部弄懂。你只需要记住一个大方向：学 *ROS 2*。后面章节里出现的所有命令和示例，都默认运行在 ROS 2 之上。

1.10 ROS 2 与 Ubuntu 版本的搭配

ROS 2 和 Ubuntu 一样，也有发行版的概念，而且两者高度绑定：官方每两年发布一个 ROS 2 的 LTS 版本，与当年度 Ubuntu 的 LTS 版本一一对应。最常用的组合如下：

Ubuntu 版本	ROS 2 发行版	说明
20.04（2020 LTS）	Foxy（2020 LTS）	较旧，已停止维护
22.04（2022 LTS）	Humble（2022 LTS）	目前最主流，教程最多
24.04（2024 LTS）	Jazzy（2024 LTS）	最新，适合新硬件
26.04（2026 LTS）	Lyrical（2026 LTS）	本书使用的组合

给初学者的建议很明确：*Ubuntu 22.04 + ROS 2 Humble* 是目前资料最全、社区最活跃、兼容性问题最少的组合。如果你的电脑比较新，或者像本章一样用 WSL 装了 26.04，那就跟着 26.04 + Lyrical 走；下面给出的安装命令也都是以 Lyrical 为例。

这里有一条最重要的原则：不要混搭。比如在 Ubuntu 20.04 上硬装 Humble，或在 22.04 上装 Foxy，往往会遇到一堆莫名其妙的编译错误，而问题根源只是版本不对。版本搭配对了，后面能少走一半弯路。等你在第二章学会打开终端之后，可以用 `lsb_release -a` 这条命令随时查看自己系统的 Ubuntu 版本。

1.11 在 WSL 里安装 ROS 2

终于到了正题。前面说过 ROS 2 和 Ubuntu 是严格配对的，所以第一步永远是确认系统版本。

这一条千万记住:**版本对不上,装是装不上的**。在 26.04 上敲 `apt install ros-humble-desktop`，得到的只会是 `Unable to locate package`，因为软件源里根本没有为这个系统编译的 Humble 包。

1.11.1 官方路线：ros2-apt-source

ROS 2 官方现在的推荐做法，是用一个叫 `ros2-apt-source` 的配置包自动帮你把软件源和 GPG 密钥都配好，不用再手工拼源地址：

自动配置 ROS2 软件源

```
sudo apt install curl -y
export ROS_APT_SOURCE_VERSION=$(curl -s https://api.github.com/repos/ros-
    infrastructure/ros-apt-source/releases/latest | grep -F "tag_name" | awk
     -F'"' '{print $4}')
curl -L -o /tmp/ros2-apt-source.deb "https://github.com/ros-infrastructure/
    ros-apt-source/releases/download/${ROS_APT_SOURCE_VERSION}/ros2-apt-
    source_${ROS_APT_SOURCE_VERSION}.$(. /etc/os-release && echo ${
    UBUNTU_CODENAME:-${VERSION_CODENAME}})_all.deb"
sudo dpkg -i /tmp/ros2-apt-source.deb
sudo apt update
```

这条路干净利落，但它有一个前提：连得上 GitHub。笔者这里就翻车了：

连不上 GitHub

```
curl: (7) Failed to connect to github.com port 443 after 3 ms: Could not
    connect to server
```

那就换国内的镜像源走。

1.11.2 国内镜像路线

思路很简单：自己把密钥和源地址写进去，跳过 GitHub。用清华源举例：

从清华镜像取密钥并转换格式

```
sudo rm -f /usr/share/keyrings/ros-archive-keyring.gpg
sudo rm -f /etc/apt/sources.list.d/ros2.list
curl -sSL https://mirrors.tuna.tsinghua.edu.cn/rosdistro/ros.key | sudo gpg
    --dearmor -o /usr/share/keyrings/ros-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/
    ros-archive-keyring.gpg] https://mirrors.tuna.tsinghua.edu.cn/ros2/
```

```
    ubuntu $(lsb_release -cs) main" | sudo tee /etc/apt/sources.list.d/ros2.
    list > /dev/null
sudo apt update
```

这里藏着一个非常值得记住的坑。如果只是把 ros.key 下载下来直接丢进 keyrings 目录，apt update 会给你这样一段：

密钥格式不对时的报错

```
Err:4 https://mirrors.tuna.tsinghua.edu.cn/ros2/ubuntu resolute InRelease
  The following signatures couldn't be verified because the public key is
    not available:
  NO_PUBKEY F42ED6FBAB17C654
Warning: The key(s) in the keyring /usr/share/keyrings/ros-archive-keyring.
    gpg
  are ignored as the file has an unsupported filetype.
Error: The repository '...' is not signed.
Error: Unable to locate package ros-lyrical-desktop
```

关键字是 unsupported filetype。新版 apt 要求密钥文件必须是二进制格式，而官网给的 ros.key 是 ASCII 文本。中间那一步 gpg --dearmor 就是干这个转换的，少了它，密钥等于没装。NO_PUBKEY 和 Unable to locate package 都是它的连锁反应。

1.11.3 安装并验证

源通了之后，装桌面版，它包含 RViz2、rqt、demos 等全套工具：

安装 ROS2 Lyrical 桌面版

```
sudo apt install ros-lyrical-desktop -y
```

这个过程会顺带把 OpenJDK、点云库 PCL、可视化库 VTK 等一大堆依赖一起装上，输出能刷几百行，看到 Setting up ros-lyrical-desktop 就说明成了。

然后配环境变量，让每个新终端都认识 ros2 命令：

配置环境变量

```
echo "source /opt/ros/lyrical/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

验证还是老一套的“你说我听”，开两个终端：

验证安装

```
# 终端 1：发布者
ros2 run demo_nodes_cpp talker

# 终端 2：订阅者
ros2 run demo_nodes_py listener
```

看到一边刷 `Publishing: 'Hello World: 1'`、另一边刷 `I heard: [Hello World: 1]`，就成功了。

1.11.4 如果两个节点互相听不见

这一步在 WSL 里失败的概率不低，而且现象很迷惑：两个节点都正常跑着，就是收不到消息。

原因出在 WSL 默认的 NAT 网络模式上。ROS 2 底层靠 DDS 通信，节点之间要靠多播互相发现，而 NAT 模式会把这个过程挡住。解决办法还是那条：在 `C:\Users\<你的用户名>\.wslconfig` 里改成镜像模式，然后 `wsl --shutdown` 重启。

.wslconfig

```
[wsl2]
networkingMode=mirrored
```

第四章还会介绍鱼香 ROS 的一键安装脚本，那是另一种更省事的装法，两条路殊途同归；无论走哪条，验证方式都是上面这段“你说我听”。

1.12 让 rviz2 正常出图

最后一个坑，也是笔者花了最多时间的一个。小海龟能正常跑，说明 WSLg 的图形转发没问题；但一敲 `rviz2`，就是这条：

rviz2 启动失败

```
Invalid parentWindowHandle (wrong server or screen) in GLXWindow::create
```

这个错误的根子在 rviz2 用的 Ogre 渲染引擎和 WSLg 默认的 Wayland 协议之间。Ogre 想按老一套 X11 的方式去创建窗口，结果拿到的窗口句柄“不是它熟悉的那个”。

1.12.1 第一招：强制走 XCB

最直接的办法，是让 Qt 程序改用更成熟的 X11 后端，绕开 Wayland 那条路：

强制使用 XCB 后端

```
export QT_QPA_PLATFORM=xcb
rviz2
```

有效的话就写进 ~/.bashrc 长期保留。不过要提醒一句，这个变量是给 rviz2 这类基于 Qt 和 Ogre 的程序用的，**不建议**全局常开，它会影响其他本来就适配良好的 Wayland 应用。

1.12.2 第二招：补图形库，并让 Mesa 用上显卡

如果第一招没用，就检查 OpenGL 这一层。这里遇到的第一个报错是包名过期：

libgl1-mesa-glx 已经不存在了

```
Error: Package 'libgl1-mesa-glx' has no installation candidate
```

libgl1-mesa-glx 这个包从 Ubuntu 23.10 起就被废弃了，现在的对应物是 libgl1 和 libglx-mesa0：

正确的包名

```
sudo apt update
sudo apt install libgl1 libglx-mesa0 libgl1-mesa-dri mesa-utils
```

装完还要多做一步。WSLg 里 Mesa 默认不一定挑中 D3D12 这条通显卡的路，可能悄悄退回 llvmpipe，也就是纯 CPU 软件渲染。软件渲染也能画，但帧率会难看得让人怀疑人生，所以显式指定驱动：

让 Mesa 走 D3D12

```
echo 'export GALLIUM_DRIVER=d3d12' >> ~/.bashrc
source ~/.bashrc
```

验证方式是把当前渲染器打出来：

查看当前使用的 OpenGL 渲染器

```
glxinfo -B | grep -E "OpenGL renderer|Device"
```

笔者的机器上输出是：

硬件加速已生效

```
    Device: D3D12 (NVIDIA GeForce RTX 4060 Laptop GPU) (0xffffffff)
OpenGL renderer string: D3D12 (NVIDIA GeForce RTX 4060 Laptop GPU)
```

看到 D3D12 加上自己的显卡型号，就说明 GPU 直通已经工作了。如果这里显示的是 llvmpipe，那说明还在软件渲染，回头检查 GALLIUM_DRIVER 有没有生效。

1.12.3 最后一招：软件渲染兜底

如果显卡这条路怎么都不通，那还有最后的退路，牺牲性能换能用：

强制软件渲染

```
LIBGL_ALWAYS_SOFTWARE=1 rviz2
```

这只适合应急，界面的卡顿会非常明显，别当长期方案。

1.13 这一路的坑，一张表收尾

把上面所有的报错和原因压缩成一张速查表，下次遇到直接查：

现象	真正原因	一句话解决
`wsl --list --online` 报 502	连不上 GitHub	关代理，或直接离线装
桌面版 ISO 装不进 WSL	ISO 是安装程序，不是根文件系统	下 `.wsl` 镜像，用 `--from-file`
`mkdir /opt` 权限拒绝	没 sudo；且路径与 D 盘无关	加 sudo，迁移回 PowerShell 做
apt 报 `dpkg lock` 权限错	没加 sudo	命令前加 sudo
一条 apt 命令什么都没装上	其中有个包名不存在，整条失败	拆开逐条装，先确认包名
中文显示成方块	缺字体	装 `fonts-noto-cjk`
装了 fcitx 打不出中文	与 WSLg 的输入法桥接冲突	卸掉，直接用 Windows 输入法
`wsl --shutdown` 找不到命令	在 Linux 侧敲了 Windows 命令	去 PowerShell，或敲 `wsl.exe`
`gedit` 后面一串分号数字	ANSI 控制序列被原样打印	无需处理，或 `2>/dev/null`
终端里敲网址报“没有那个文件或目录”	把网址当命令了	先 `git clone`
snap 卡在 0%	国内访问 snap 商店不稳定	重启 snapd，或用 PPA 装 deb 版
桌面起不来，日志写 asserts failed	缺 `XDG_SESSION_TYPE`	写进 `environment.d`
`NO_PUBKEY` / unsupported filetype	密钥是文本格式	`gpg --dearmor` 转二进制
两个节点互相听不见	NAT 挡了 DDS 多播	`networkingMode=mirrored`
rviz2 报 Invalid parentWindowHandle	Ogre 与 Wayland 不兼容	`QT_QPA_PLATFORM=xcb`
rviz2 卡成幻灯片	退回 llvmpipe 软件渲染	`GALLIUM_DRIVER=d3d12`

1.14 这条路值不值得走

写到这里，笔者对 WSL 的看法是这样的。

它的好处很实在：不用装虚拟机，开机两秒进系统；`/mnt/c` 让 Windows 和 Ubuntu 共用一套文件；WSLg 让 rviz2、小海龟这些图形工具的窗口直接摆在 Windows 桌面上，截图往笔记里一贴就行；再加上它和 VS Code 的 WSL 扩展配合得很好，写代码的体验比装双系统强。

它的代价也很实在：没有快照，弄坏了只能重装（或者靠 `wsl --export` 自己备

份）；图形相关的坑特别多，因为多了一层 WSLg 中转；版本非常新的时候（比如 26.04 配 GNOME 50），社区方案还没跟上，得自己看日志一点点试。

所以笔者的建议是：日常学习和写代码就用 WSL，轻快省事；等到项目真的需要实时性、需要接真实的机器人硬件，再考虑把它搬到一台独立的 Ubuntu 机器上。

最后回到正题。如果你只是想把 ROS 2 跑起来，其实根本不用折腾桌面那一大段。按本章的顺序，装好发行版、配好字体、装完 ROS 2、解决 rviz2 的渲染问题，就足够开始第四章那些内容了。至于 GNOME 桌面，那属于“我想让它更好看”的部分，和学 ROS 2 没有关系。

另外一个不该被忽略的收获：这一整章的报错，几乎全部来自同一个根源，就是在两套系统之间搞混了层次。命令该在哪边敲、路径属于哪个文件系统、包名是哪个系统里的，把这三件事分清楚，一半的坑根本不会踩。

ROS2 官方教程入口

```
https://docs.ros.org
# 文档结构: Installation / Tutorials / Guides / Concepts / Contributing
# 新手从 Tutorials 里的 Beginner: CLI tools 开始
```

万一英文读着吃力，鱼香 ROS 社区维护了一份与官方同步的中文文档镜像，可以对照着看。查文档时留意页面顶部的版本标识，跟着自己装的那个版本走，别拿 Lyrical 的教程去操作 Humble 的环境。

1.15 本书的学习路线

这一章建立的是“地图”，也顺手把“地基”打好了。接下来的章节才是“走路”：

- **第二章** Ubuntu 常用指令：学会打开终端，掌握文件、目录、权限等最常用的命令，这是之后所有操作的地基；
- **第三章** Ubuntu 环境下编程：看看 C++ 和 Python 程序在 Linux 里怎么编译、怎么运行，和 Windows 下有什么不同；
- **第四章** ROS 2 基础：从节点、话题、服务这些核心概念开始，配合小例子逐个验证；
- 最后把各部分串起来，完成小型机器人程序，体会完整的工作流。

如果你现在对“Ubuntu 到底是什么、WSL 怎么用、ROS 能干什么”已经有了大致印象，那这一章的任务就完成了。翻开第二章，我们从打开终端开始。

第二章　Ubuntu 常用指令

2.1　打开终端

第一件事是学会打开终端。常用的终端打开方式无外乎两种：

- 使用快捷键 Ctrl+Alt+T
- 文件夹中右击打开 *open in Terminal*

2.2　Linux 中的常用命令

2.2.1　ls 命令

就是 list 的缩写，通过 ls 命令不仅可以查看 *linux* 文件夹包含的文件，而且可以查看文件权限 (包括目录、文件夹、文件权限)，查看目录信息等等。

ls 命令常用参数

```
ls -a  列出目录所有文件，包含以.开始的隐藏文件
ls -A  列出除.及..的其它文件
ls -r  反序排列
ls -t  以文件修改时间排序
ls -S  以文件大小排序
ls -h  以易读大小显示
ls -l  列出文件的权限、所有者、文件大小等详细信息
```

实例：

按易读方式按时间反序排序，并显示文件详细信息

```
ls -lhrt
```

按大小反序显示文件详细信息

```
ls -lrS
```

列出当前目录中所有以”t” 开头的目录的详细内容

```
ls -l t*
```

列出文件绝对路径（不包含隐藏文件）

```
ls | sed "s:^:`pwd`/:"
```

列出文件绝对路径（包含隐藏文件）

```
find $pwd -maxdepth 1 | xargs ls -ld
```

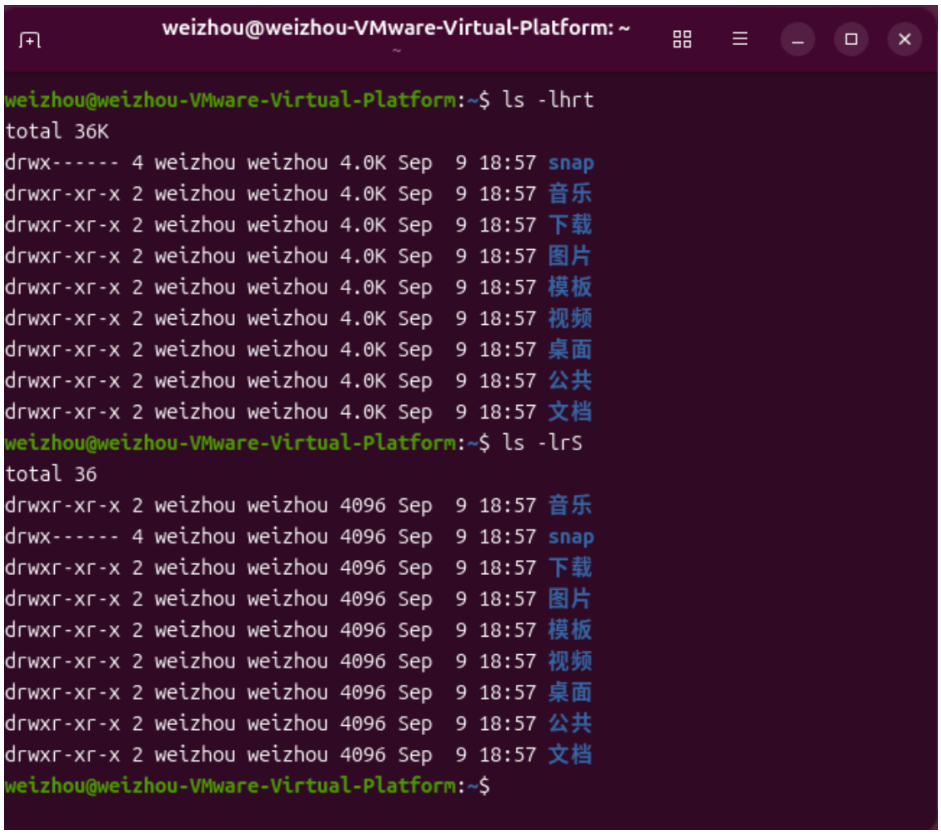


图 2.1: ls 命令示例

2.2.2 cd 命令

cd(changeDirectory) 命令语法：

cd(changeDirectory) 命令语法：

```
cd [目录名]
```

说明：切换当前目录至 dirName。

示例：

进入要目录

```
cd /
```

进入”home” 目录

```
cd ~
```

进入上一次工作路径

```
cd -
```

把上个命令的参数作为 cd 参数使用。

```
cd !$
```

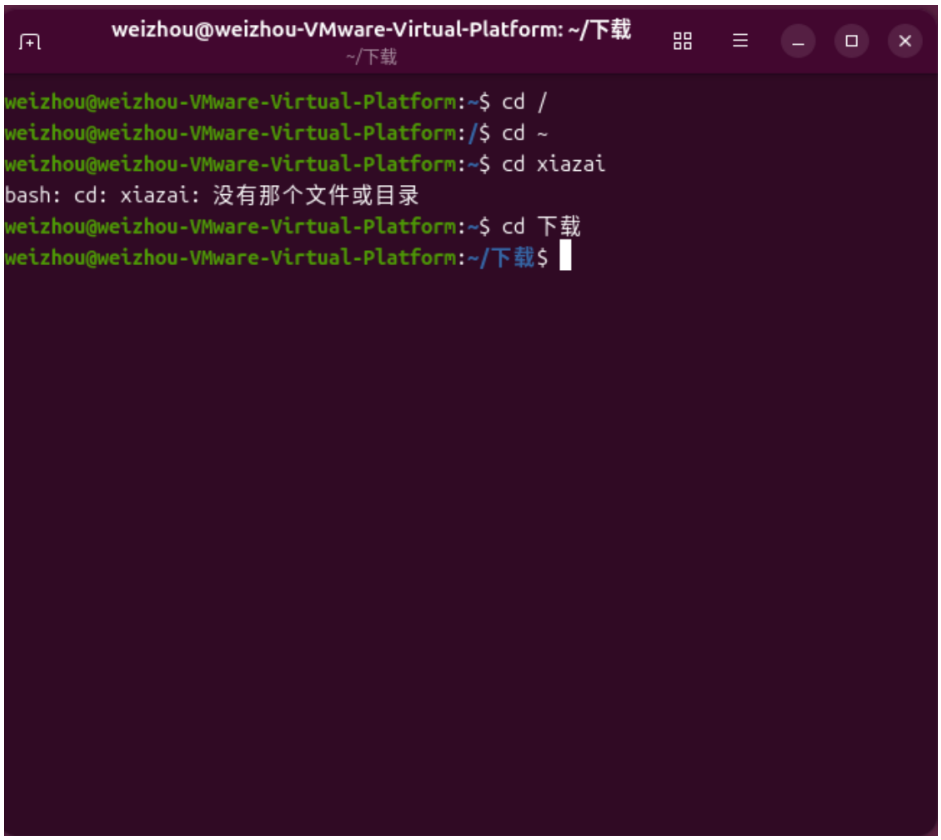


图 2.2: cd 命令示例

2.2.3 pwd 命令

pwd 命令用于查看当前工作目录路径。它是初学者最常用来确认 “我现在在哪” 的命令。

查看当前路径

```
pwd
```

查看软链接的实际路径

```
pwd -P
```

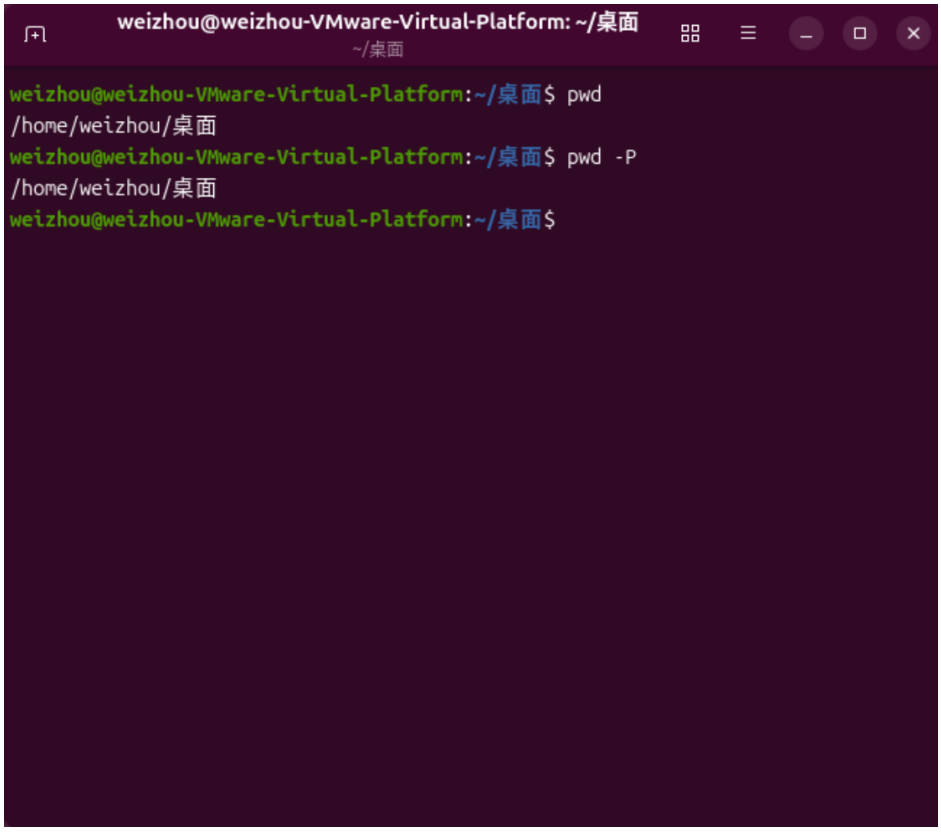


图 2.3: pwd 命令示例

2.2.4 mkdir 命令

mkdir 命令用于创建文件夹。

mkdir 常用参数

```
mkdir -m    对新建目录设置存取权限
mkdir -p    若路径中某些目录尚不存在，系统会自动建立(一次建多级)
```

实例：

当前工作目录下创建名为 t 的文件夹

```
mkdir t
```

在 tmp 下创建 path test/t1/t, 若目录不存在则一并建立

```
mkdir -p /tmp/test/t1/t
```

2.2.5 rm 命令

删除一个或多个文件或目录。如果不加 *-r* 选项，rm 不会删除目录。删除文件通常仍可以恢复，但删除目录需谨慎。

rm 常用参数

```
rm -i    删除前逐一询问确认
rm -r    递归删除(连同子目录一起删除)
rm -f    强制删除，不提示
```

实例：

删除任何.log 文件, 删除前逐一询问

```
rm -i *.log
```

删除 test 子目录及其中所有内容, 不一一确认

```
rm -rf test
```

删除以 - 开头的文件 (用 – 结束选项)

```
rm -- -f*
```

2.2.6 cp 命令

将源文件复制到目标文件，或将多个源文件复制到目标目录。

cp 常用参数

```
cp -i    目标文件已存在时提示是否覆盖
cp -r    复制目录及目录内所有内容
cp -a    保持原文件的时间等属性
cp -s    建立符号链接(快捷方式)
```

实例：

复制 a.txt 到 test 目录, 保持时间, 存在则提示

```
cp -ai a.txt test
```

为 a.txt 建立一个链接 (快捷方式)

```
cp -s a.txt link_a.txt
```

2.2.7 mv 命令

移动文件，或修改文件 / 目录名。当第二个参数是目录时，将第一个参数（可以是多个）移动到该目录。

mv 常用参数

```
mv -i     目标已存在时提示是否覆盖
```

实例：

将文件 test.log 改名为 test1.txt

```
mv test.log test1.txt
```

把多个文件移动到根的 test3 目录

```
mv log1.txt log2.txt log3.txt /test3
```

移动当前目录下所有文件到上一级目录

```
mv * ../
```

2.2.8 cat 命令

cat 主要有三大功能：一次显示整个文件；从键盘创建一个新文件；将几个文件合并成一个文件。

cat 常用参数

```
cat -b     对非空输出行号
cat -n     输出所有行号
```

实例：

一次显示整个文件

```
cat filename
```

从键盘创建一个文件 (只能新建, 不能编辑已有文件)

```
cat > filename
```

把 log2012.log 内容加上行号后写进 log2013.log

```
cat -n log2012.log log2013.log
```

反向列示文件 (与 cat 相反)

```
tac log.txt
```

2.2.9 more 命令

功能类似于 cat，但会一页一页显示，方便逐页阅读。按空格键往下翻一页，按 *b* 键往回翻一页，按 *q* 键退出。

more 常用参数

```
more +n     从第 n 行开始显示
more -n     定义屏幕大小为 n 行
more -s     把连续多个空行显示为一行
```

实例：

显示文件中从第 3 行起的内容

```
more +3 text.txt
```

借助管道让 ls -l 每次只显示 5 行

```
ls -l | more -5
```

2.2.10 less 命令

less 与 more 类似，但使用 less 可以随意前后浏览文件，more 只能向前。而且 less 在查看之前不会加载整个文件，更适合大文件。

less 常用操作

```
/字符串      向下搜索字符串
?字符串      向上搜索字符串
n            重复前一个搜索
N            反向重复前一个搜索
空格          滚动一页
b            向后翻一页
-q           退出
```

实例：

查看进程信息并通过 less 分页显示 (带行号)

```
ps -aux | less -N
```

查看多个文件 (按 n 下一个, p 前一个)

```
less 1.log 2.log
```

2.2.11 head 命令

显示文件的开头内容，默认显示前 10 行。

head 常用参数

```
head -n <行数>     显示的行数
head -c <字节数>   显示前多少个字节
```

实例：

显示 1.log 文件中前 20 行

```
head 1.log -n 20
```

显示最后 10 行 (行数为负数表示从后往前数)

```
head -n -10 t.log
```

2.2.12 tail 命令

显示文件末尾内容，常用来查看日志文件。

tail 常用参数

```
tail -f     循环读取(适合查看动态增加的日志)
tail -n <行数>   显示行数(从后往前)
```

实例：

循环读取逐渐增加的文件内容

```
tail -f ping.log
```

2.2.13 which、whereis、locate、find 命令

这四个命令都能用来找文件，但原理和适用场景各不相同：

- **which** 在 PATH 中查找可执行命令的位置，返回第一个匹配结果。
- **whereis** 只搜索程序名相关的二进制、帮助和源码文件，速度快。
- **locate** 基于系统数据库查找文件，速度很快，但新建文件可能找不到。
- **find** 实际遍历硬盘查找文件，最准确但最慢。

实例：

查看 ls 命令是否存在及其位置

```
which ls
```

查找 locate 程序的相关文件

```
whereis locate
```

查找与 pwd 相关的所有文件

```
locate pwd
```

在当前目录查找以.log 结尾的文件

```
find ./ -name '*.log'
```

查找 /opt 目录下权限为 777 的文件

```
find /opt -perm 777
```

查找更改时间在 10 日以前的文件并删除 (无提示)

```
find . -type f -mtime +10 -exec rm -f {} \;
```

2.2.14 grep 命令

强大的文本搜索命令，全称 Global Regular Expression Print，即全局正则表达式搜索。它在文件或多个文件中搜索字符串模板，结果输出到标准输出，不改变原文件内容。

grep 常用参数

```
grep -c    计算符合样式的列数
grep -i    忽略大小写
grep -l    只列出文件名
grep -n    显示匹配行所在行号
grep -R    递归查找文件夹
```

实例：

查找指定进程

```
ps -ef | grep svn
```

从文件夹递归查找以 grep 开头的行, 只列出文件

```
grep -lR '^grep' /tmp
```

显示包含 ed 或 at 内容的行为

```
grep -E 'ed|at' test.txt
```

2.2.15 wc 命令

wc(word count) 用于统计指定文件中字节数、单词数、行数，并将统计结果输出。

wc 常用参数

```
wc -c    统计字节数
wc -l    统计行数
wc -m    统计字符数
wc -w    统计单词数
```

实例：

统计文件的行数单词数字节数

```
wc text.txt
```

统计输出的行数

```
cat test.txt | wc -l
```

2.2.16 chmod 命令

用于改变 Linux 系统文件或目录的访问权限。权限用 r(读)、w(写)、x(执行) 表示，对应数字为 4、2、1。每一文件有三组权限：属主、属组、其他用户。

chmod 常用参数

```
chmod -R    递归处理目录及子目录下所有文件
```

权限代号：

权限数字对应

```
r = 4    读
w = 2    写
x = 1    执行
```

实例：

增加文件 t.log 所有用户的可执行权限

```
chmod a+x t.log
```

属主读写执行 (7), 属组读执行 (5), 其他执行 (1)

```
chmod 751 t.log
```

将 test 目录及子目录所有文件添加可读权限

```
chmod u+r,g+r,o+r -R text/
```

2.2.17 tar 命令

用来打包和解压文件。注意 tar 本身只打包不压缩，压缩与解压是调用其他工具（如 gzip、bzip2）完成的。常用的两招是 *tar zcvf* 打包压缩、*tar zxvf* 解压。

tar 常用参数

```
-c    建立新的压缩文件
-x    从压缩包中解压
-t    显示压缩文件中的内容
-z    支持 gzip 压缩
-j    支持 bzip2 压缩
-v    显示操作过程
-f    指定压缩文件名
```

实例：

将文件全部打包成 tar 包

```
tar -cvf log.tar log1.log log2.log
```

将 /etc 下所有文件和目录打包并用 gz 压缩

```
tar -zcvf /tmp/etc.tar.gz /etc
```

查看刚打包的文件内容 (必须加 z, 因为用 gzip 压缩)

```
tar -ztvf /tmp/etc.tar.gz
```

解压

```
tar -zxvf filename.tar.gz
```

2.2.18 ps 命令

ps(process status) 用来查看当前运行的进程状态，一次性查看。如需动态连续结果，用 *top*。

ps 常用参数

```
ps -A     显示所有进程
ps -e     显示所有进程(等同 -A)
ps -f     显示进程间的关系(完整格式)
ps -aux   显示所有包含其他信息的进程
```

实例：

显示当前所有进程环境变量及进程间关系

```
ps -ef
```

与 grep 联用查找某进程

```
ps -aux | grep apache
```

2.2.19 top 命令

动态显示当前系统正在执行的进程信息，包括进程 ID、CPU 占用率、内存占用率等。交互界面里按 q 退出。

top 常用参数

```
top -p <进程号>     指定进程显示
top -n <次数>       循环显示次数
```

top 交互命令

```
P     按 CPU 使用率排序
M     按内存使用率排序
q     退出
```

2.2.20 free 命令

显示系统内存使用情况，包括物理内存、交换区内存 (swap) 和内核缓冲区内存。

free 常用参数

```
free -m     以 MB 为单位显示
free -g     以 GB 为单位显示
free -s <秒>  每隔多少秒持续显示
```

实例：

显示内存使用情况

```
    free -m
```

2.2.21 df 命令

显示磁盘空间使用情况，获取硬盘已占用多少、还剩多少等信息。

df 常用参数

```
df -h     以易读方式显示
df -T     列出文件系统类型
df -a     全部文件系统列表
```

实例：

以易读方式列出所有文件系统及其类型

```
    df -haT
```

2.2.22 du 命令

查看文件和目录的磁盘使用空间，与 df 不同，du 针对的是具体文件或目录。

du 常用参数

```
du -h     以易读方式显示文件大小
du -s     仅显示总计
du -a     显示目录中所有文件大小
```

实例：

以易读方式显示文件夹内及子文件夹大小

```
    du -h scf/
```

显示几个文件各自占用空间并统计总和

```
du -hc test/ scf/
```

2.2.23 ln 命令

为文件在另一个位置建立一个链接。软链接 (*ln -s*) 类似于 Windows 里的快捷方式，可跨文件系统、可用于目录；硬链接 (*ln*) 以文件副本形式存在，但同一文件系统内才可创建，且不占用实际空间。无论改哪一处，其余链接都会同步变化。

ln 常用参数

```
ln -s    软链接(符号链接)
ln -v    显示详细处理过程
```

实例：

给文件创建软链接并显示操作信息

```
ln -sv source.log link.log
```

2.2.24 date 与 cal 命令

date 显示或设定系统的日期与时间；cal 显示公历（阳历）日历。

date 常用格式

```
%Y    年份(四位数)
%m    月份(01-12)
%d    日期(01-31)
%H    小时(00-23)
%M    分钟(00-59)
```

实例：

显示指定日期格式

```
date +%Y%m%d
```

显示下一天

```
date -d "+1 day" +%Y%m%d
```

显示 2013 年每个月的日历

```
cal -y 2013
```

将星期一作为第一列, 显示前中后三月

```
cal -3m
```

2.2.25 kill 命令

发送指定信号给相应进程。默认发送 SIGTERM(15) 终止指定进程，若无法终止可用 *-KILL*（发送 SIGKILL，9）强制结束进程。用 ps 或 jobs 查看进程号；root 用户可影响所有用户的进程，普通用户只能影响自己。

kill 常用参数

```
kill -l           列出全部信号名称
kill -9 <PID>     强制结束进程
```

实例：

先查找进程 pro1, 然后强制杀掉

```
kill -9 $(ps -ef | grep pro1)
```

2.3 鱼香 ROS 的指令

上面的是 Linux 的常用指令，下面一个指令非常重要，是咱们使用 ROS 系统可以会的指令，极大节省时间：

一键安装

```
wget http://fishros.com/install -O fishros && . fishros
```

这条指令来自我关注的博主“鱼香 ROS”，非常好用。效果如下图：显然，包含了咱

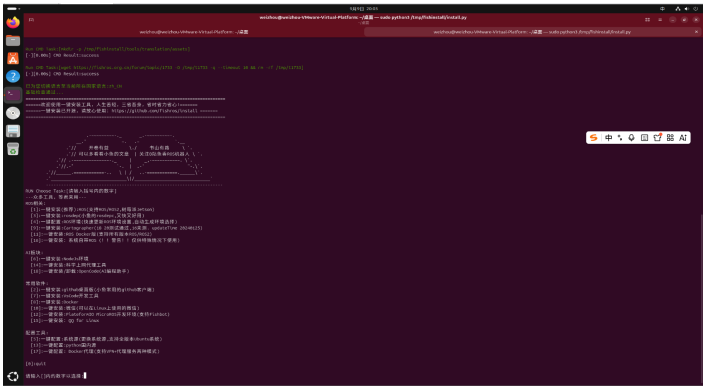


图 2.4: 小鱼的一键安装命令

们要用的几乎所有工具。

2.4 用自然语言来使用 Unbuntu

AI 时代，古法编程的艺术仍在流传，但 Vibe Coding 已经成为主流。我选择在保留自己读写代码能力的同时，先装一个 npm, 再装上 opencode，然后美美使用中文使用 Unbuntu（doge）

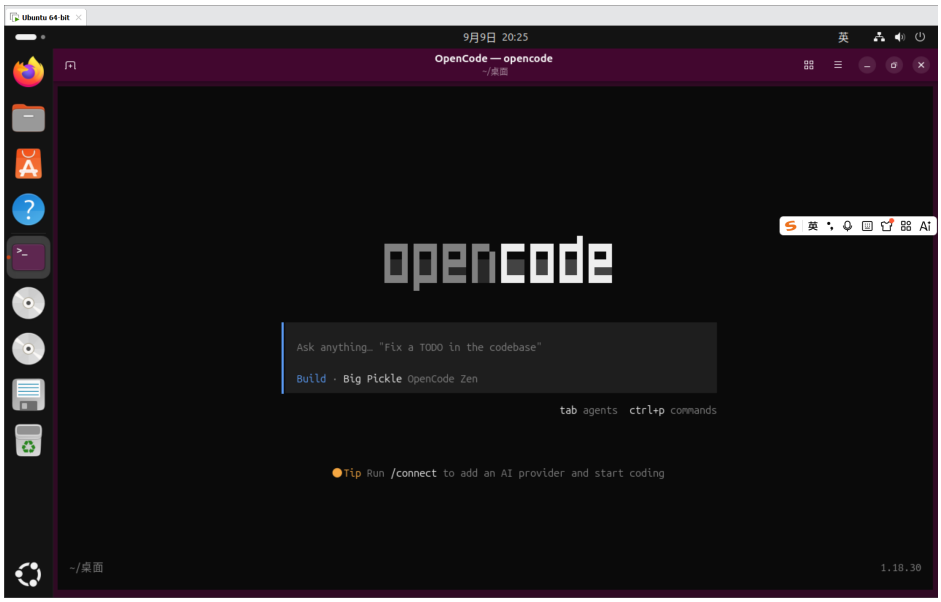


图 2.5: Unbuntu 中的 opencode

第三章　Ubuntu 环境下编程

做机器人最常用的两门语言就是 C++ 和 Python，同时这两门语言也是编程语言流行度排行榜数一数二的。然后这两门咱们都学过。C++ 是课上教的，python 语法极其简单，不赘述了。

一般用 Linux 的都是程序员里的大佬，都是大佬了，一般就喜欢拿记事本 grdit 写代码，因为不用打开编辑器，全程可以在命令行终端里执行。Linux 里面还有逆天 Vim，但此等极道帝兵显然不是我等炼气期小辈能够掌握的。以后编程估计都在 VS code 里完成，但是先看一下 Ubantu 里面程序的运行和编写和 Windows 下有何不同。

3.1　Hello weijianguizhou!

3.1.1　C++

打开终端，创建一个 d2lros2 文件夹接着在文件夹下创建 chapt1 文件夹。然后输入这行命令：

用 gedit 创建并打开 cpp 文件

```
gedit hello_weijianguizhou.cpp
```

输入：

cpp 程序

```
# include <iostream>
using namespace std;
int main(){
    cout<<"Hello weijianguizhou!"<<endl;
    return 0;
}
```

保存代码，关闭 gedit。

C++ 代码必须经过编译构建之后才能运行

C++ 的编译工具使用的是 g++，默认是不安装的，所以我们需要手动安装一下 g++。

安装 g++ 编译器

```
sudo apt install g++
```

接着在终端输入下面的指令对刚刚的代码进行编译。

编译 C++

```
g++ hello_weijianguizhou.cpp
```

接着使用 ls 指令，应该可以看到一个叫做 a.out 的文件。恭喜，现在可以运行了。

运行 C++

```
./a.out
```

可以看出区别了。Windows 系统中的可执行文件为.exe，而 Linux 系统中的可执行文件为.out.

真尼玛麻烦。

3.1.2 python

然后输入这行命令：

用 gedit 创建并打开 py 文件

```
gedit hello_weijianguizhou.py
```

输入

py 程序

```
print("Hello weijianguizhou!")
```

保存代码，关闭 gedit。直接运行：

运行 py

```
python3 hello_weijianguizhou.py
```

下面看一下 python 和 C++ 对比。由此看来，python 相比 C++ 有如下优势：

一、python 程序比 C++ 简单。C++ 要 6 行才能搞定的程序我 python 只要 1 行就能搞定，此乃一胜。

二、python 在命令行中的运行比 C++ 简单。C++ 最短要三行，而我 python 只要一行，此乃二胜。

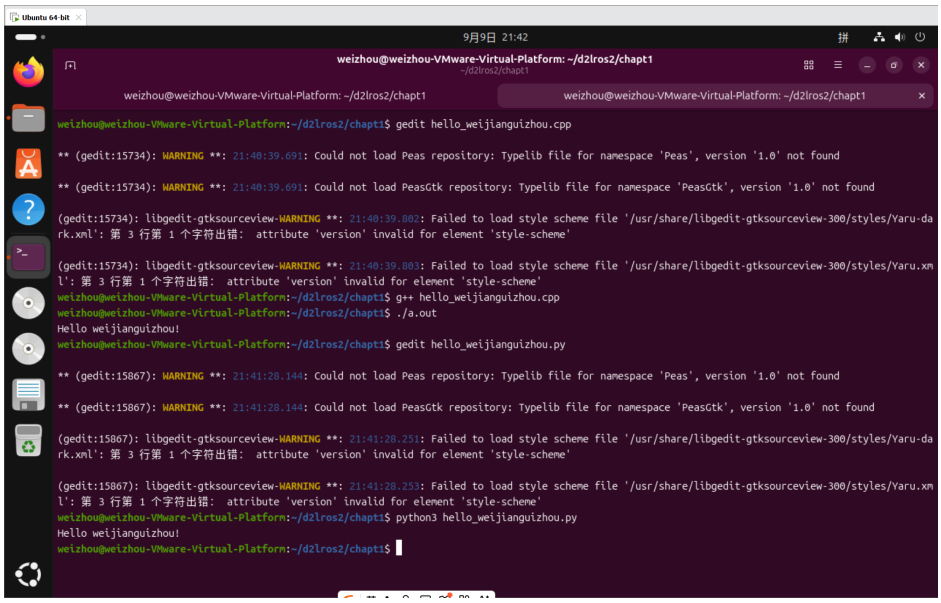


图 3.1: python 和 C++ 对比

三、python 语法更接近自然语言。此乃三胜。

由此看来，python 大获全胜..... 吗???

并非啊！虽然 python 好用，但是它，慢啊!!!

为啥？

因为 C++ 是编译执行，而 python 是解释执行。

3.2 什么是编译执行和解释执行

语言本身用解释型语言还是编译型语言来划分就是不准确的。

设计语言的人，也不会考虑先考虑什么型，而是在设计中发现某种方式更好，便会选择使用。语言属于 XX 型是呈现给大家的一个印象。

编程语言，是程序员们操控电脑以实现各种功能的主要方式，而解释执行与编译执行，是计算机编程语言的两种执行方式。

更多的时候，某个语言使用到了编译执行也用到了解释执行。

现在来笼统解释一下。

- 编译：将源代码一次性转换为机器码的过程（机器码有保存为文件，下次运行的时候直接运行机器码）
- 解释：将源代逐行转换为机器码并运行的过程（机器码并没有保存下来）
- 编译执行（编译器）：将一段程序直接翻译成机器码 (对于 C/C++ 这种非跨平台的语言) 或者中间码 (Java 这种跨平台语言，需要 JVM 再将字节码编译成

机器码)。编译执行是直接将所有语句都编译成了机器语言，并且保存成可执行的机器码。执行的时候，是直接进行执行机器语言，不需要再进行解释/编译。

- 解释执行（解释器）：在执行程序时，再将中间码（例如 Java 的字节码通过 JVM 解释成机器码）一行行的解释成机器码进行执行。这个运行过程是解释一行，执行一行。

执行编译过程的程序叫做编译器。

执行解释过程的程序叫做解释器。

王垠所说的编译型语言和解释型语言的本质都是解释执行的。

编译型和解释型的语言之所以可以说相同，是因为这些语言从文本形式的源代码到最终产生的结果，所经历的过程是一样的。

根本区别是运行时，解释型需要将程序解释成机器码来运行，并没有保存机器码，是在运行过程中进行。

而编译型在运行之前就已经让编译器给程序编译成机器码了。

这也是为什么编译运行会比解释运行快的根本原因。

在知乎上看到一个比喻，非常好。

编译相当于做好了一桌子菜，可以直接开吃了。

而解释就相当于吃火锅，需要一边煮一边吃。

那么自然，吃的效率也会低一些。

第四章　ROS2 基础

4.1　ROS2 的下载

首先启动 WSL 中的 Ubuntu，打开终端，输入下面的指令。

一键安装指令

```
wget http://fishros.com/install -O fishros && . fishros
```

输入密码，在选项界面选择 1-一键安装 ROS，接着根据你的情况选择是否更换系统源，接着等待一会就会让你选择要安装的 ROS2 版本了。这里选择 humble 版本的 ROS2 即可。

接着会问你安装桌面版还是基础版，我们选择桌面版，包含可视化工具，嗯，我就喜欢可视化工具，如果是在树莓派上装可以使用基础版。

安装完成后输入 ros2 如果看到下面的界面则安装成功。

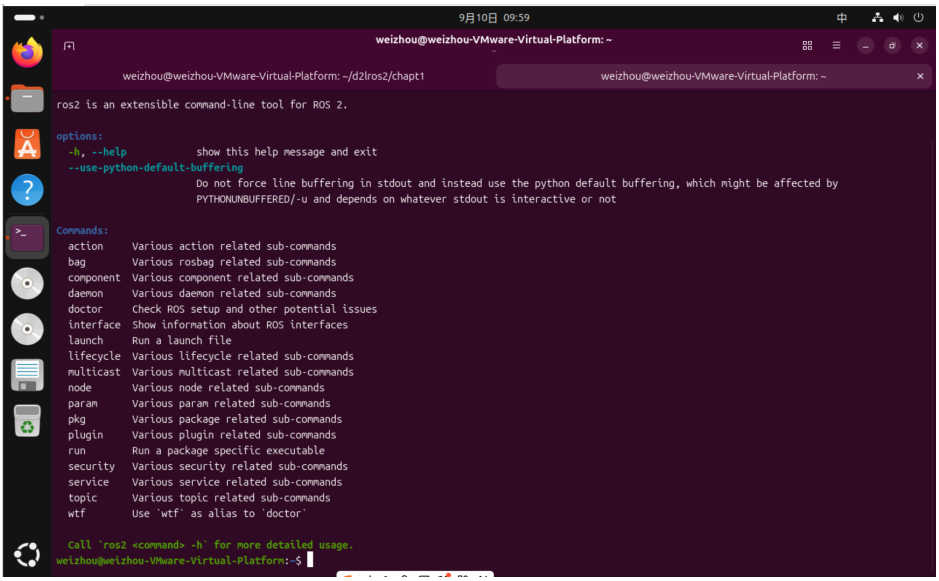


图 4.1: ROS2 成功安装界面

那要是我瞎玩，给 ROS2 环境弄乱了。该怎么办呢？删掉重装就好了呀。

一键卸载 ROS2

```
sudo apt remove ros-humble-*
sudo apt autoremove
```

我们都知道安装软件要设置安装目录。那么 ROS2 装在哪里了呢？ROS 安装的默认目录在/opt/ros/下，根据版本的名字进行区分。安装的是 humble 版本的 ROS，所以安装目录在/opt/ros/humble 下。

4.2 ROS2 体验

4.2.1 你说我听

游戏内容：很简单，我们启动两个节点，一个节点负责发消息 (说)，一个节点负责收消息（听）。

1. 启动一个终端，快捷键 `Ctrl+Alt+T`。
2. 在终端中启动倾听者：`ros2 run demo_nodes_py listener`
3. 再启动一个新终端，快捷键 `Ctrl+Alt+T`。
4. 在新终端中启动说话者：`ros2 run demo_nodes_cpp talker`

观察一下现象，talker 节点每数一个输，倾听节点每一次都能听到一个，是不是很神奇。

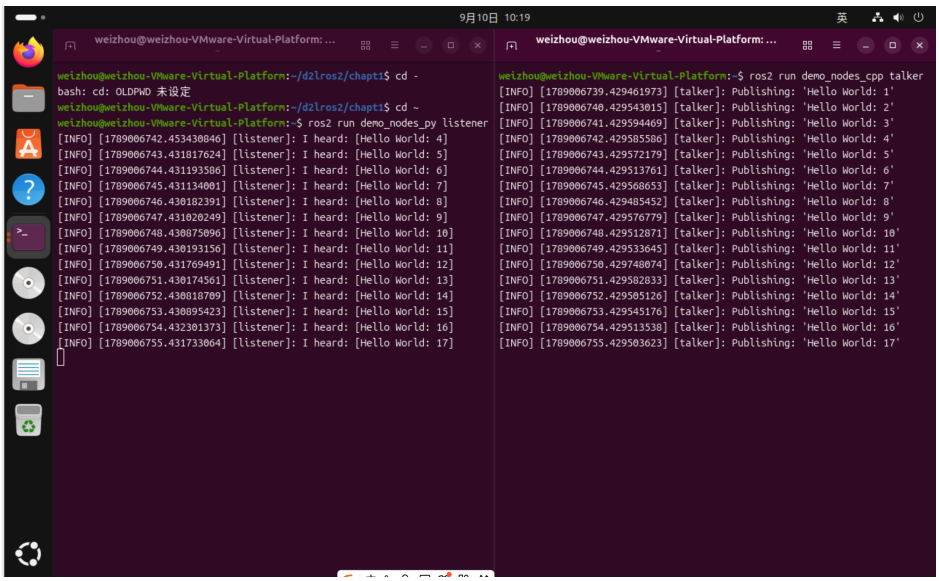


图 4.2: 你说我听

4.2.2 涂鸦乌龟

打开终端 Ctrl+Alt+T, 输入下面的指令

启动海龟画图

```
ros2 run turtlesim turtlesim_node
```

如下图所示：

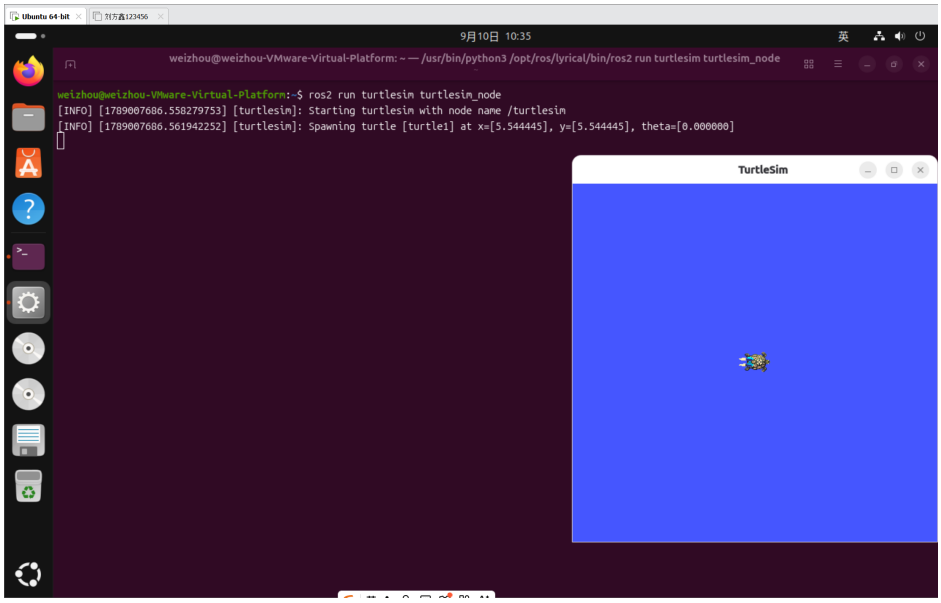


图 4.3: 海龟模拟器

点一下原来的终端输入 Ctrl+Shift+T 打开一个新的标签页输入

启动海龟操控

```
ros2 run turtlesim turtle_teleop_key
```

4.2.3 RQT 可视化

保持前面两个游戏在运行状态，打开终端，输入 rqt。打开之后选择插件 Introspection / Node Graph，刷新一下，即可看见两个节点间的通信关系。

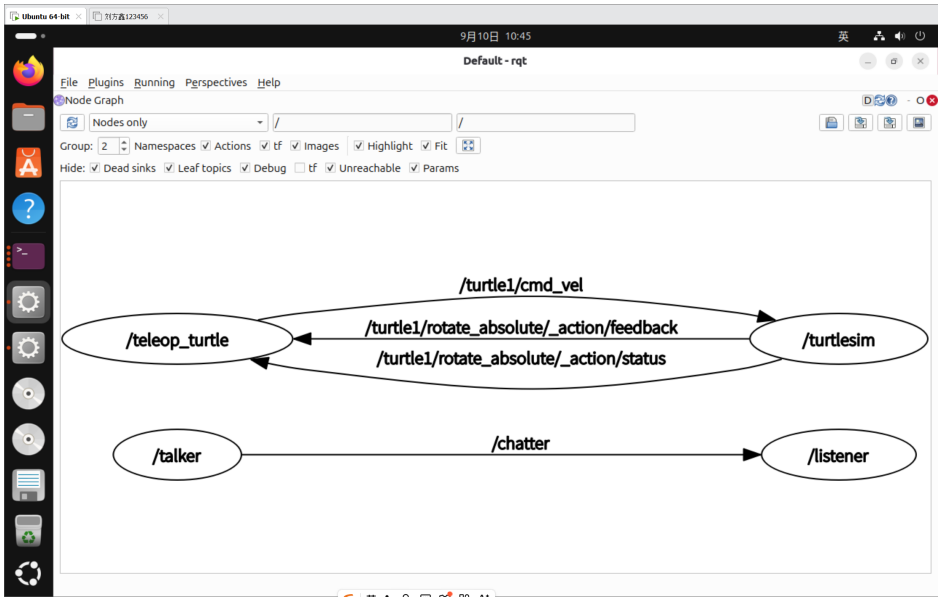


图 4.4: RQT Node Graph 可视化

4.3 ROS2 系统架构

前面两个小游戏你已经玩过了，节点能跑起来，rqt 里也能看到它们之间的连线。不过这些都还停留在“会用”的层面。这一节我们把 ROS2 拆开，从整体框架上看一看它到底由哪几层构成，以及为什么要这样分层。

4.3.1 分层架构总览

ROS2 的整体结构可以用一张“千层饼”式的图来描述，从上到下依次是：应用层、ROS2 客户端库、客户端库核心 rcl、中间件抽象层 RMW、DDS 实现层、操作系统层。相邻两层之间是双向的调用关系：上层向下层提出请求，下层把结果和事件回传给上层。

图 4.5 中两侧的括号标出了各层的归属：从应用层到客户端库核心 rcl，这三层是 ROS2 官方自己实现的；中间件抽象层 RMW 是统一接口；最下面的 DDS 实现层则来自第三方。接下来我们自下而上，一层一层地看。

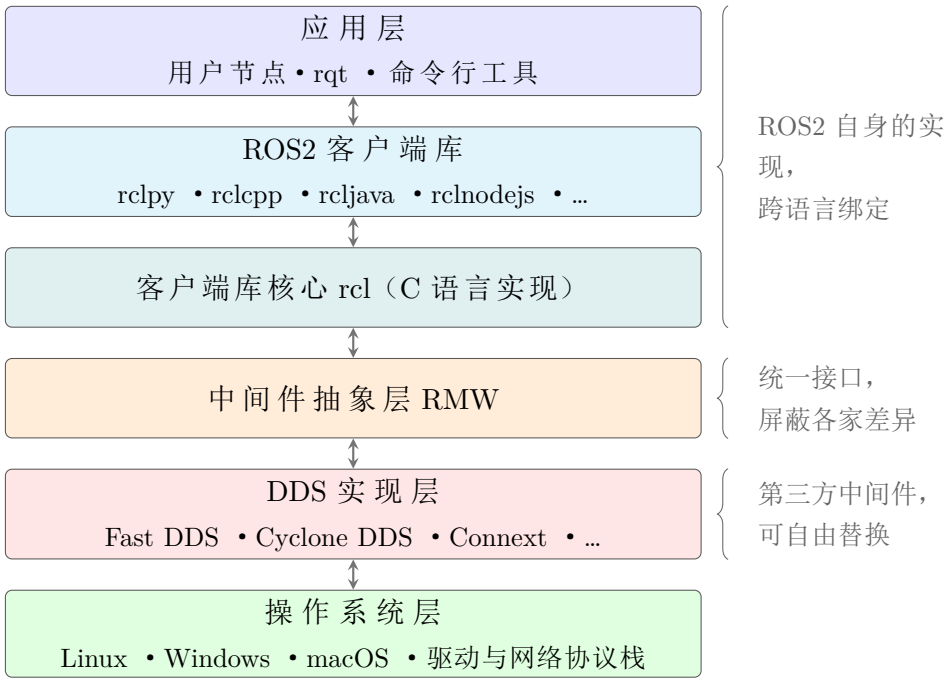


图 4.5: ROS2 的分层架构。自下而上依次是操作系统层、DDS 实现层、中间件抽象层 RMW、客户端库 RCL 以及最顶端的应用层，相邻两层之间为双向调用关系

4.3.2 操作系统层

这一层最好理解。ROS2 自己并不是一个操作系统，它只是跑在 Linux、Windows 或 macOS 之上的一个中间件框架。真正去驱动硬件、管理进程、收发底层网络数据包这些脏活累活，全部交给操作系统来完成。也正因为如此，ROS2 才能在这么多平台上跑起来：只要那个平台提供的系统调用和网络协议栈足够完整，上面的各层就可以原封不动地搬过去。

顺带说一句，我们在第一章里折腾 WSL 里的 Ubuntu，其实就是在给上面的几层准备一块合适的“地基”。

4.3.3 DDS 实现层

想弄懂这一层，你得先回答两个问题：DDS 到底是什么？以及为什么 ROS2 里会同时存在好几套 DDS 实现？

DDS 是什么

DDS，全称 Data Distribution Service，中文叫“数据分发服务”，是由对象管理组 OMG（Object Management Group）在 2003 年发布、2007 年修订的一套分布式系统通信标准。

它的通信方式和 ROS 里的话题发布订阅几乎一模一样：一方把数据“发布”到

某个主题上，关心这个主题的另一方“订阅”后就能收到，双方不需要事先知道对方是谁。除此之外，DDS 还提供了相当丰富的服务质量（QoS）策略，用来规定通信的可靠性、持久性、传输方式等要求。

正因为设计理念如此契合，ROS2 干脆不再自己另造一套通信协议，而是直接把 DDS 拿来当作底层的通信支柱。

为什么会有多套 DDS 实现

DDS 只是一份标准文档，标准本身不能运行，得有人照着它写出真正的程序，这些程序就叫“DDS 实现”。目前常见的实现有 eProsima 的 Fast DDS、Eclipse 的 Cyclone DDS、RTI 的 Connext DDS 等。它们遵守同一份标准，但侧重点各不相同：有的偏重性能，有的体积小巧，有的面向商用场景提供技术支持。

既然标准是统一的，那多准备几套实现就不会给上层添麻烦，想换随时换。至于这些差异是怎么被“抹平”的，就要看下面这一层了。

4.3.4 中间件抽象层 RMW

RMW 是 ROS Middleware 的缩写，也就是中间件抽象层。它的作用可以用一句话概括：**给上面一个统一的接口，把下面各家的差异藏起来**。

为什么不直接把 DDS 暴露给上层呢？因为直接用 DDS 其实挺麻烦的。创建一次通信要设置分区、主题名称、发现模式、消息类型等一大堆参数，还要记得在不用的时候逐项清理。这些琐碎而重复的配置如果让每个写节点的同学自己处理，代码会变得又臭又长。

于是 ROS2 在这中间加了一层 RMW：它定义好一套固定的接口，无论底下换成 Fast DDS 还是 Cyclone DDS，上层看到的 API 都完全一样。那些繁琐的配置也就顺理成章地在 RMW 和 RCL 里统一完成了。

查看与切换 DDS 实现

```
# 查看当前使用的 DDS 实现
ros2 doctor --report | grep middleware

# 通过环境变量切换（需要系统里已安装对应的 DDS）
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
```

4.3.5 客户端库 RCL 与语言绑定

RCL 是 ROS Client Library 的缩写，也就是 ROS2 客户端库。它建立在 RMW 之上，把 ROS 里的节点、话题、服务、参数、Action 这些概念包装成一套好用的

API。

这里有个细节值得留意：RCL 的底层实现是用 C 语言写的，因为 C 是众多高级语言的“共同祖先”，基于它再去封装其它语言会轻松很多。所以官方的 rclpy（Python）和 rclcpp（C++）都是直接建立在这套 C 实现之上的。这也是为什么后面我们用两种语言写节点时，代码结构会如此相似。

社区还用同样的方式把 ROS2 搬到了更多语言和平台上，比如在 Android 应用或网页里也能和 ROS2 通信：

表 4.1: ROS2 的常见多语言客户端库

语言	客户端库	项目地址
Python	rclpy	https://github.com/ros2/rclpy
C++	rclcpp	https://github.com/ros2/rclcpp
Java	rcljava	https://github.com/esteve/ros2_java
Rust	rclrust	https://github.com/ros2-rust/ros2_rust
Node.js	rclnodejs	https://github.com/RobotWebTools/rclnodejs
Go	rclgo	https://github.com/juaruipav/rclgo
Lua	rcllua	https://github.com/jbbjarnason/rcllua
Kotlin	rclkin	https://github.com/ros2java-alfred/ros2_kotlin
Swift	rclswift	https://github.com/atyshka/ros2_swift
C#	rclcs	https://github.com/RobotecAI/ros2cs

4.3.6 应用层

最上面就是我们自己写代码、以及 rqt、命令行工具这些开发工具所在的层了。本书后面写的所有节点代码，其实都住在这里。它们不需要关心底下的通信是怎么实现的，只要调用客户端库提供的 API 就好，这也正是分层带来的好处。

4.4 ROS2 中间件 DDS 架构

4.4.1 中间件

中间件是一种独立的系统软件或服务程序，分布式应用软件借助这种软件在不同的技术之间共享资源。中间件位于客户机/ 服务器的操作系统之上，管理计算机资源和网络通讯。是连接两个独立应用程序或独立系统的软件。相连接的系统，即使它们具有不同的接口，但通过中间件相互之间仍能交换信息。执行中间件的一个关键途径是信息传递。通过中间件，应用程序可以工作于多平台或 OS 环境。



图 4.6: 这是在说什么，怎么一个字都听不懂（）

4.5 DDS 和 ROS2 架构

ROS2 为每家 DDS 供应商都开发了对应的 `DDS_Interface` 即 DDS 接口层，然后通过 DDS Abstract 抽象层来统一 DDS 的 API。

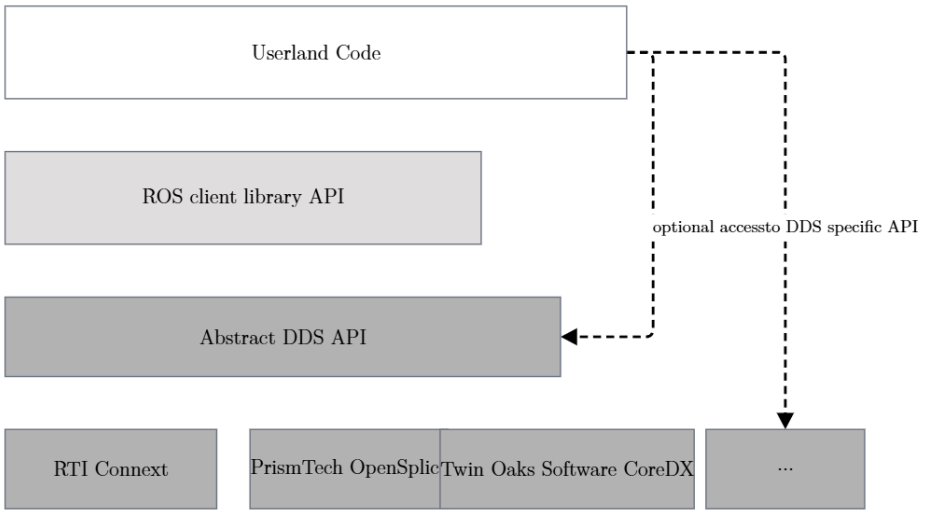


图 4.7: ROS2DDS 架构

4.6 DDS 通信模型

核心模型 DCPS：以数据为中心的发布订阅。DCPS 模型通过主题实现发布者与订阅者的解耦，核心构成包括：

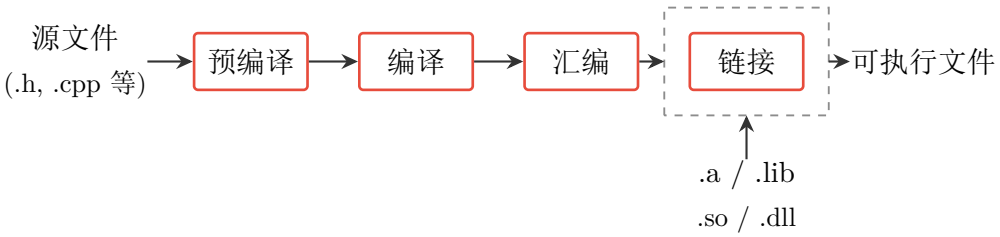
- 域（Domain）：作为逻辑隔离的通信平面，只有相同域 ID 下的节点才能互相发现和通信，解决多系统同网络下的隔离问题。
- 主题（Topic）：是连接发布者与订阅者的逻辑纽带，对数据做分类标记，是发布和订阅匹配的唯一依据，发布者只需要发布对应主题的数据，所有订阅该主题的节点都会自动收到数据。
- 发布者/数据写入器（Publisher/DataWriter）：由发布者创建 DataWriter，实际负责数据的发送。
- 订阅者/数据读取器（Subscriber/DataReader）：由订阅者创建 DataReader，实际负责接收对应主题的数据。

这种模式下发布者和订阅者不需要知道对方的地址信息，仅通过主题完成匹配，能够轻松构建复杂的多对多通信关系，实现系统的高度解耦。

4.7 ROS2 的节点

4.7.1 使用 g++ 编译 ROS2 节点

程序编译一般需要经预处理、编译、汇编和链接几个步骤。在实际应用中，有些公共代码需要反复使用，就把这些代码编译成为“库”文件。在链接步骤中，链接器将从库文件取得所需的代码，复制到生成的可执行文件中，这种库称为静态（链接）库，其特点是可执行文件中包含了库代码的一份完整拷贝，缺点是被多次使用就会多份冗余拷贝。还有一种库，就是程序在开始运行后调用库函数时才被载入，这种库独立于现有的程序，其本身不可执行，但包含着程序需要调用的一些函数，这种库称为动态（链接）库（*Dynamic Link Library*）。在 widows 平台下，静态链接库是.lib 文件，动态库文件是.dll 文件。在 linux 平台下，静态链接库是.a 文件，动态链接库是.so 文件。



静态库、动态库的区别来自【链接阶段】如何处理库，
链接成可执行程序，分别称为静态链接方式、动态链接方式。

图 4.8: 编译链接流程